Chicago_LMP

```r
library(tidyr)
library(ggplot2)
library(MASS)
library(ggdensity)
library(data.table)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:data.table':
##
##     between, first, last

## The following object is masked from 'package:MASS':
##
##     select

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
#Load data
data_PJM <- fread('PJM Data.csv')
data_Chicago <- fread('Data Chicago.csv')
data_Milwaukee <- fread('Data Milwaukee.csv')
data_CorpusChristi<- fread('Data Corpus Christi.csv')
```

```r
#Exploring data
names(data_PJM)
```

```
##  [1] "datetime_beginning_utc" "datetime_beginning_ept" "pnode_id"
##  [4] "pnode_name"             "voltage"                "equipment"
##  [7] "type"                   "zone"                   "system_energy_price_rt"
## [10] "total_lmp_rt"           "congestion_price_rt"    "marginal_loss_price_rt"
## [13] "system_energy_price_da" "total_lmp_da"           "congestion_price_da"
## [16] "marginal_loss_price_da" "spread"                 "date"
## [19] "hour"                   "am_pm"                  "day"
## [22] "day_of_year"            "month_num"              "month"
## [25] "year"                   "day_of_week"            "weekend"
## [28] "isHoliday"              "holiday"
```

```r
#Units of observation
class(data_PJM$datetime_beginning_utc)
```

```
## [1] "character"
```

```r
#Identify initial date
min(data_PJM$date)
```

```
## [1] "2021-01-01"
```

```r
#Unique price_node names
head(unique(data_PJM$pnode_name),5)
```

```
## [1] "PJM"       "BRANDONSH" "BRUNSWICK" "COOKSTOWN" "DOVER"
```

```r
#number of different pricing nodes
length(unique(data_PJM$pnode_name))
```

```
## [1] 339
```

```r
#Interesting fields to compares
data_PJM[, .(date, congestion_price_rt)]
```

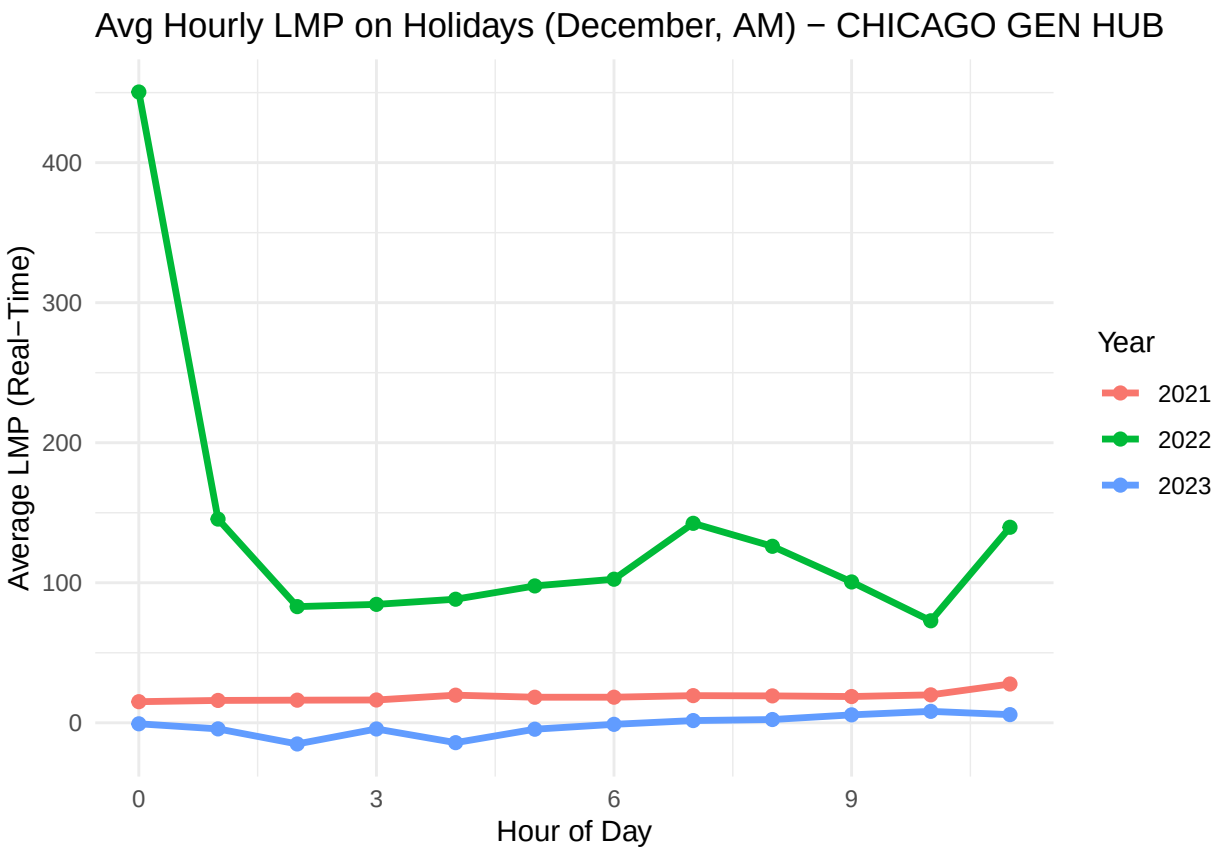
```
##                date congestion_price_rt
##              <IDat>               <num>
##       1: 2021-01-31           -0.037294
##       2: 2021-01-31           -1.106667
##       3: 2021-01-31           -4.904167
##       4: 2021-01-31           -8.830000
##       5: 2021-01-31           -6.325000
##      ---
## 8722358: 2023-12-31            0.372500
## 8722359: 2023-12-31            0.165000
## 8722360: 2023-12-31            0.039167
## 8722361: 2023-12-31            0.482417
## 8722362: 2023-12-31            0.370000
```

```r
library(dplyr)
#dataset of all observation on a holiday
holiday_PJM <- data_PJM[isHoliday == 'TRUE']
```

```r
avg_lmp_chicago_holiday = holiday_PJM[month == 'Dec' &
pnode_name == 'CHICAGO GEN HUB' &
am_pm == 'AM',
.(average_lmp_rt = mean(total_lmp_rt)), by = .(hour, year)] %>%
.[order(year)]
```

```r
#Plot to visulize data
library(ggplot2)

ggplot(avg_lmp_chicago_holiday, aes(x = hour, y = average_lmp_rt, color = factor(year))) +
  geom_line(size = 1.2) +
  geom_point(size = 2) +
  labs(title = "Avg Hourly LMP on Holidays (December, AM) - CHICAGO GEN HUB",
       x = "Hour of Day", y = "Average LMP (Real-Time)",
       color = "Year") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
#2022 shows the highest average locational marginal price at the Chicago gen hub node on holidays
#The data is being averaged over total_lmp_rt. This could be based on the winter storm chicago may have
```

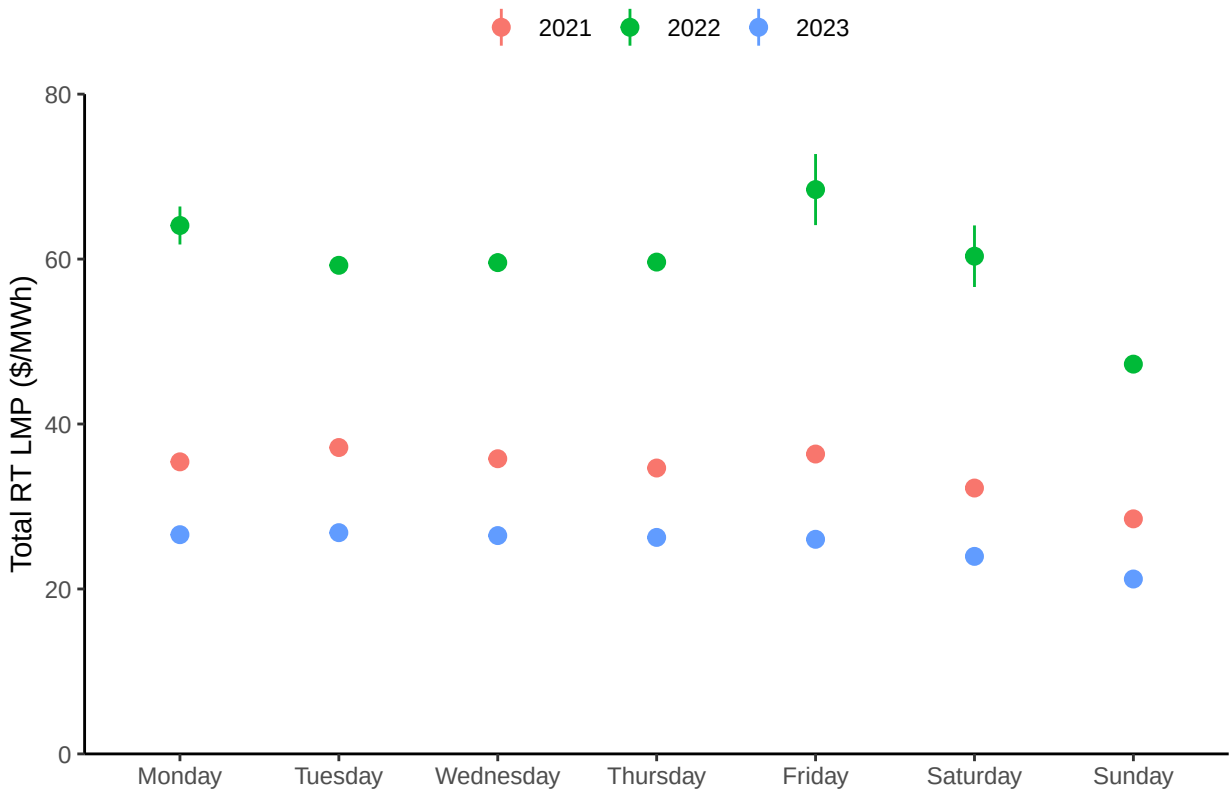
```
#Introduction to plotting
```

```
#creating data_CGH
data_CGH = data_PJM[pnode_name == 'CHICAGO GEN HUB']
#Number of observations refect the number of rows
nrow(data_CGH)
```

```
## [1] 26280
```

```
data_CGH$day_of_week <- factor(data_CGH$day_of_week,
levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"))

ggplot(data_CGH[order(day_of_week)], aes(x=day_of_week,y=total_lmp_rt, color =
    factor(year))) +
    stat_summary() +
    labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
    coord_cartesian(ylim = c(0, 80)) +
    scale_y_continuous(expand = c(0,0)) +
    theme_classic() +
    theme(legend.position = 'top')
```

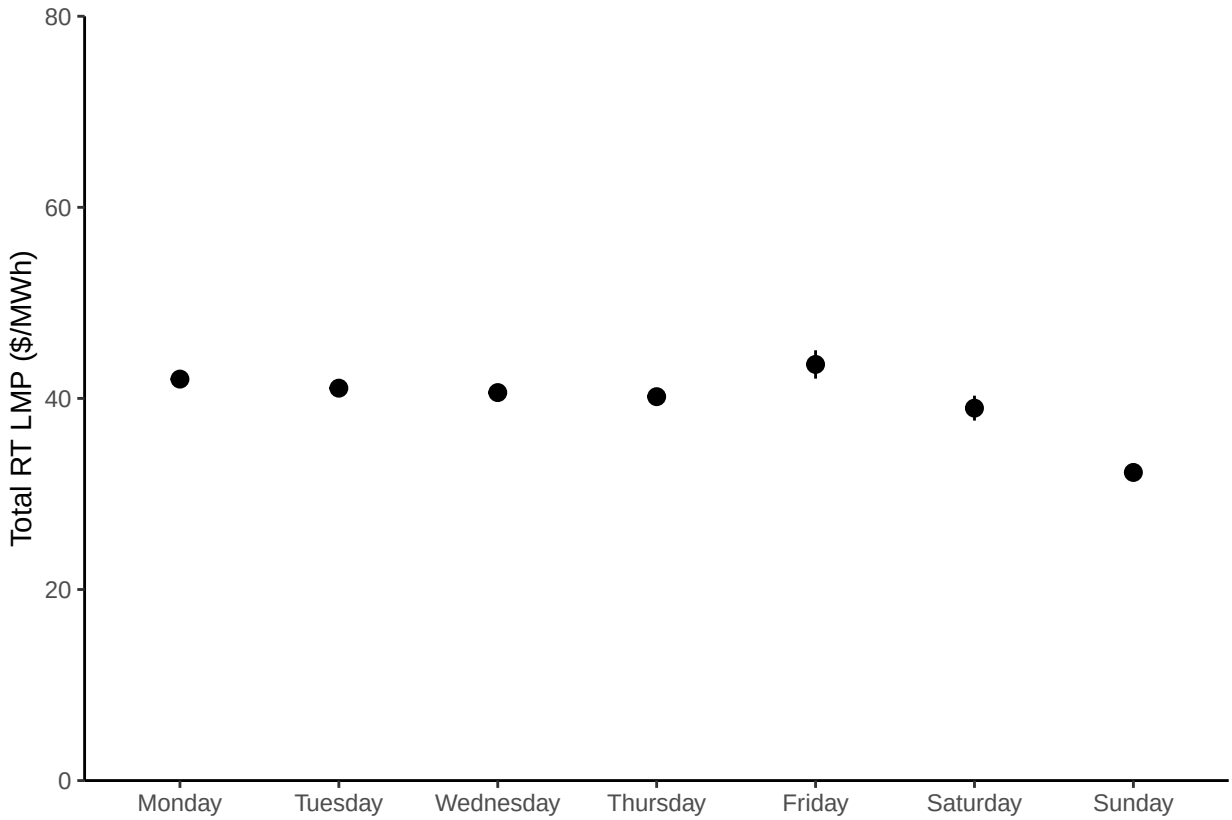
```
## No summary function supplied, defaulting to `mean_se()`
```




```
#We can see the total lmp_rt per a given day across 2021-2023. We can see that consistently had the #hi
```

```
#remove color = factor(year)
data_CGH$day_of_week <- factor(data_CGH$day_of_week, levels =
c("Monday","Tuesday","Wednesday","Thursday","Friday","Saturday",
"Sunday"))
ggplot(data_CGH[order(day_of_week)], aes(x=day_of_week,y=total_lmp_rt)) +
stat_summary() +
labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
coord_cartesian(ylim = c(0, 80)) +
scale_y_continuous(expand = c(0,0)) +
theme_classic() +
theme(legend.position = 'top')
```

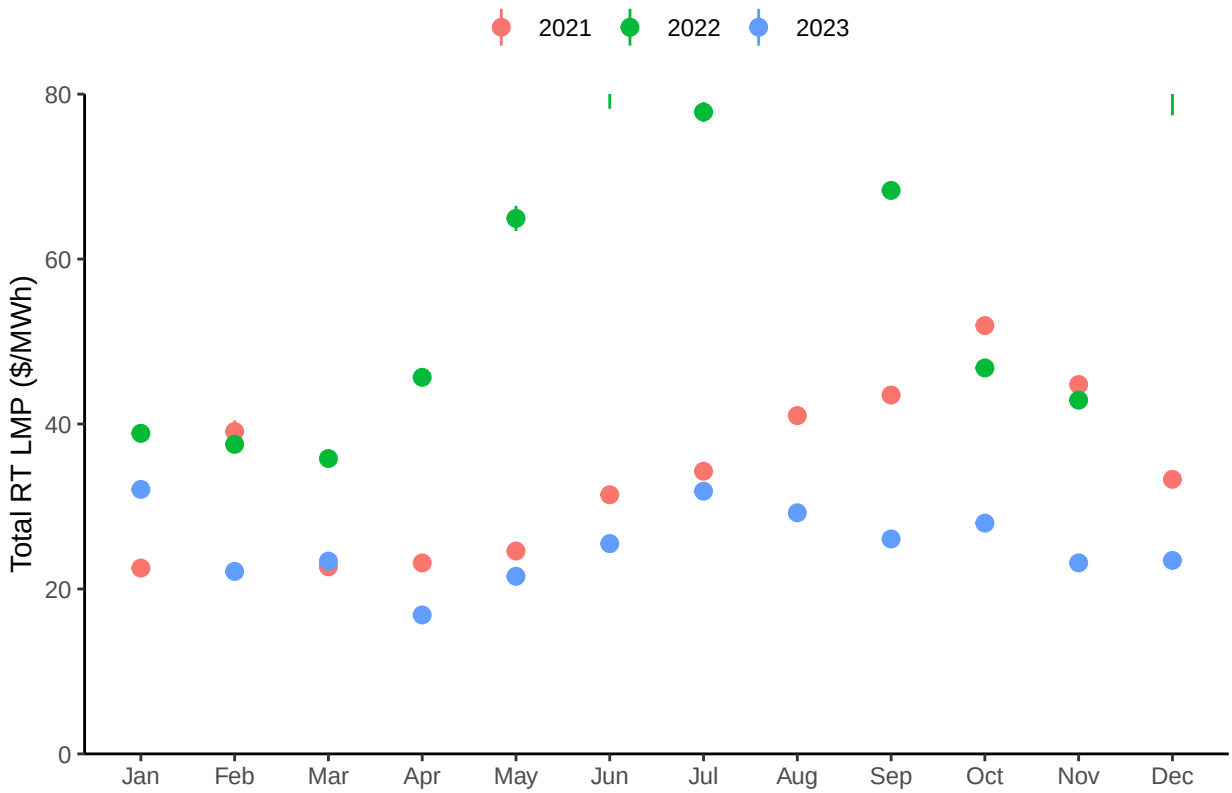
```
## No summary function supplied, defaulting to `mean_se()`
```




```
#When color = factor(year) is removed it looks like we are seeing the average across all 3 years
```

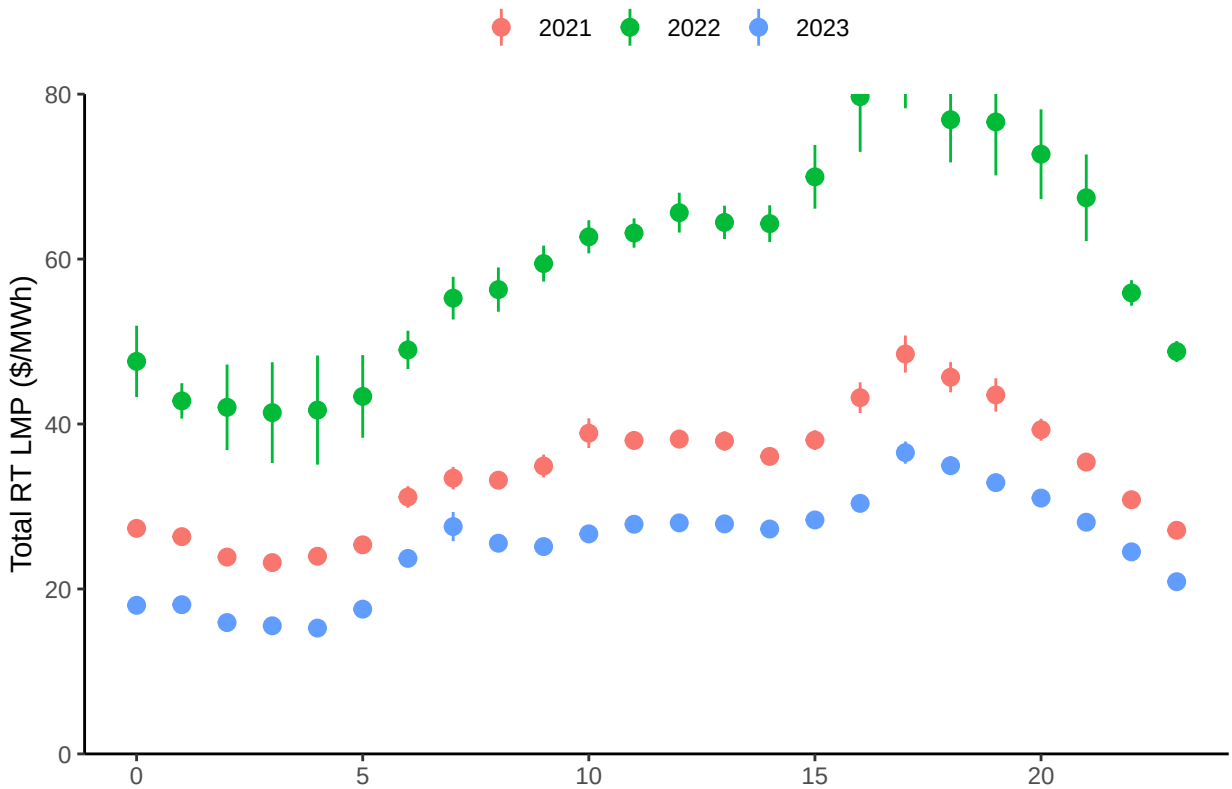
```
# By month
data_CGH$month_name <- factor(data_CGH$month, levels =
  c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"))

ggplot(data_CGH[order(month)], aes(x = month_name, y=total_lmp_rt, color =
factor(year))) +
stat_summary() +
labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
coord_cartesian(ylim = c(0, 80)) +
scale_y_continuous(expand = c(0,0)) +
theme_classic() +
theme(legend.position = 'top')
```

```
## No summary function supplied, defaulting to `mean_se()`
```




```r
#by hour
ggplot(data_CGH, aes(x=hour, y=total_lmp_rt,
        color = factor(year))) +
        stat_summary() +
        labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
        coord_cartesian(ylim = c(0, 80)) +
        scale_y_continuous(expand = c(0,0)) +
        theme_classic() +
        theme(legend.position = 'top')
```

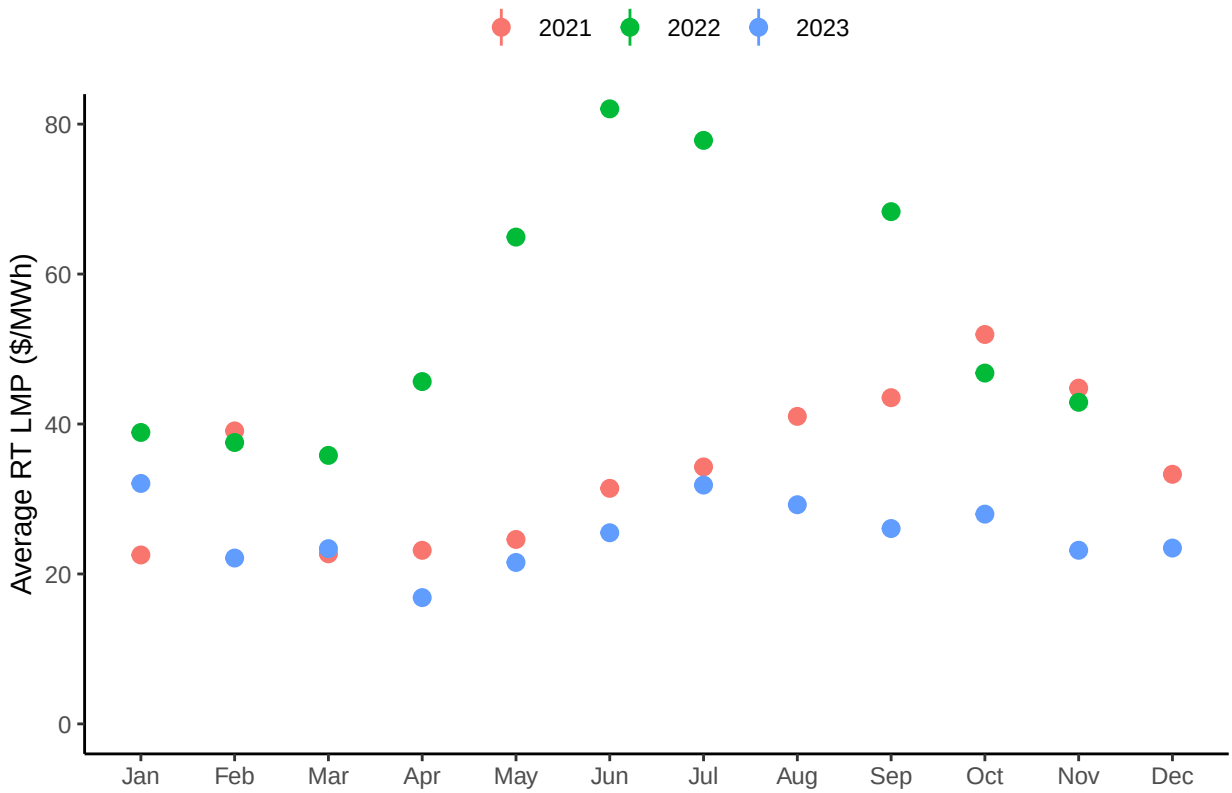
```
## No summary function supplied, defaulting to `mean_se()`
```




```
avg_lmp_chicago <- data_CGH[, .(average_lmp_rt = mean(total_lmp_rt)),
                                by = .(month, year)]

avg_lmp_chicago$month_name <- factor(avg_lmp_chicago$month, levels =
  c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"))

ggplot(avg_lmp_chicago[order(month_name)],
       aes(x = month_name,
           y = average_lmp_rt,
           color = factor(year))) +
      stat_summary() +
      labs(color = '',x = '', y = 'Average RT LMP ($/MWh)') +
      coord_cartesian(ylim = c(0,80)) +
      theme_classic() +
      theme(legend.position = 'top')
```

```
## No summary function supplied, defaulting to `mean_se()`
```

```
## Warning: Removed 36 rows containing missing values or values outside the scale range
## (`geom_segment()`).
```




```
#Chicago Summer '22 saw above higher temperatures #(https://www.weather.gov/lot/Summer_August_2022_Clim
#Also the russia-ukraine war could have been a factor that impact crude oil price and electricity is #g
```

```
#pivoting data total_lmp_rt, and total_lmp_da
library(tidyr)
library(ggplot2)
data_CGH_long <- data_CGH[, .(year, month, hour, day_of_week, total_lmp_rt,total_lmp_da)] %>%
  pivot_longer(cols = !c(year, month, hour, day_of_week),
               names_to = 'lmp_type',
               values_to = 'values')
```

```
#This goes and makes two columns into rows by creating a new column lmp type and putting
#total_lmp_rt and total_lmp_da as values
```

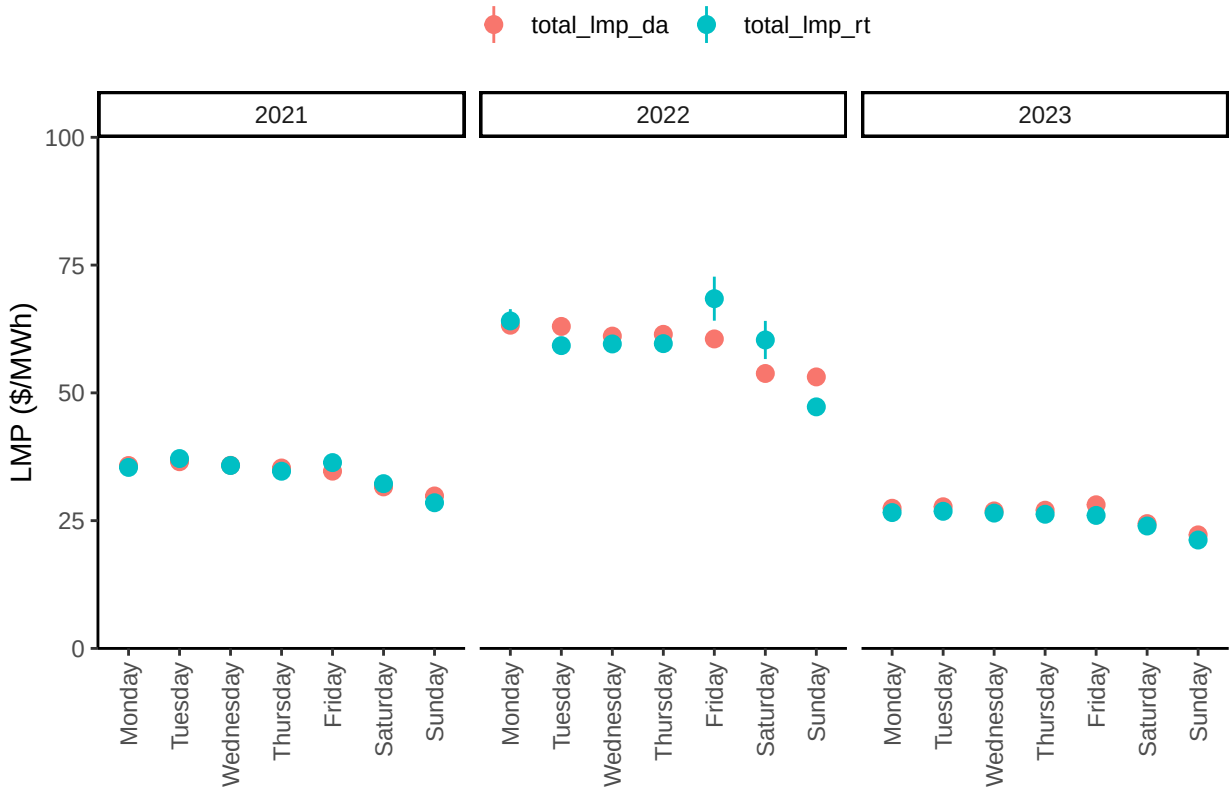
```
head(data_CGH_long)
```

```
## # A tibble: 6 x 6
##    year month  hour day_of_week lmp_type     values
##   <int> <chr> <int> <fct>       <chr>         <dbl>
## 1  2021 Jan      23 Sunday      total_lmp_rt   20.0
## 2  2021 Jan      23 Sunday      total_lmp_da   18.9
## 3  2021 Jan      22 Sunday      total_lmp_rt   21.7
## 4  2021 Jan      22 Sunday      total_lmp_da   20.3
## 5  2021 Jan      21 Sunday      total_lmp_rt   22.9
## 6  2021 Jan      21 Sunday      total_lmp_da   21.1
```

```
setDT(data_CGH_long) #go from object to data table

ggplot(data_CGH_long, aes(x=day_of_week,y=values, color = factor(lmp_type))) +
        stat_summary() +
```

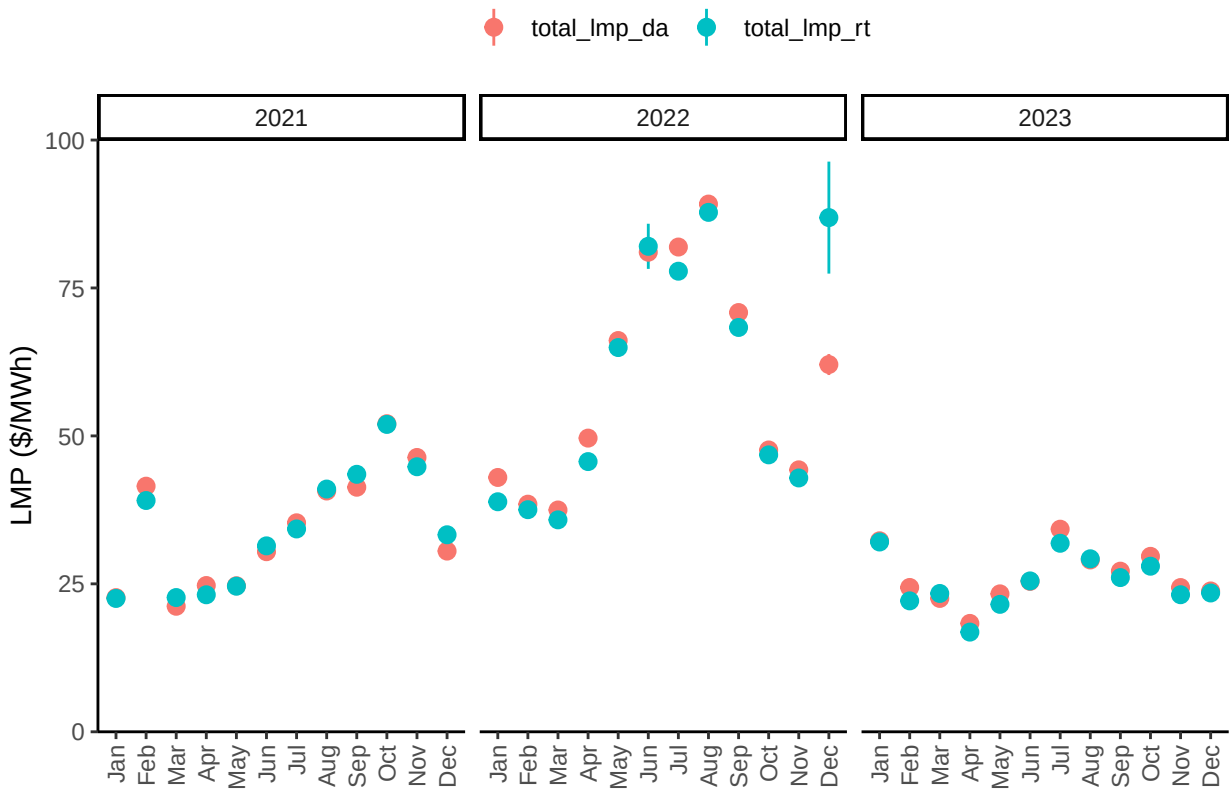
```
        labs(color = '', x = '', y = 'LMP ($/MWh)') +
        coord_cartesian(ylim = c(0, 100)) +
        scale_y_continuous(expand = c(0,0)) +
        theme_classic() +
        theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
        facet_wrap(~year)
```

```
## No summary function supplied, defaulting to `mean_se()`
## No summary function supplied, defaulting to `mean_se()`
## No summary function supplied, defaulting to `mean_se()`
```

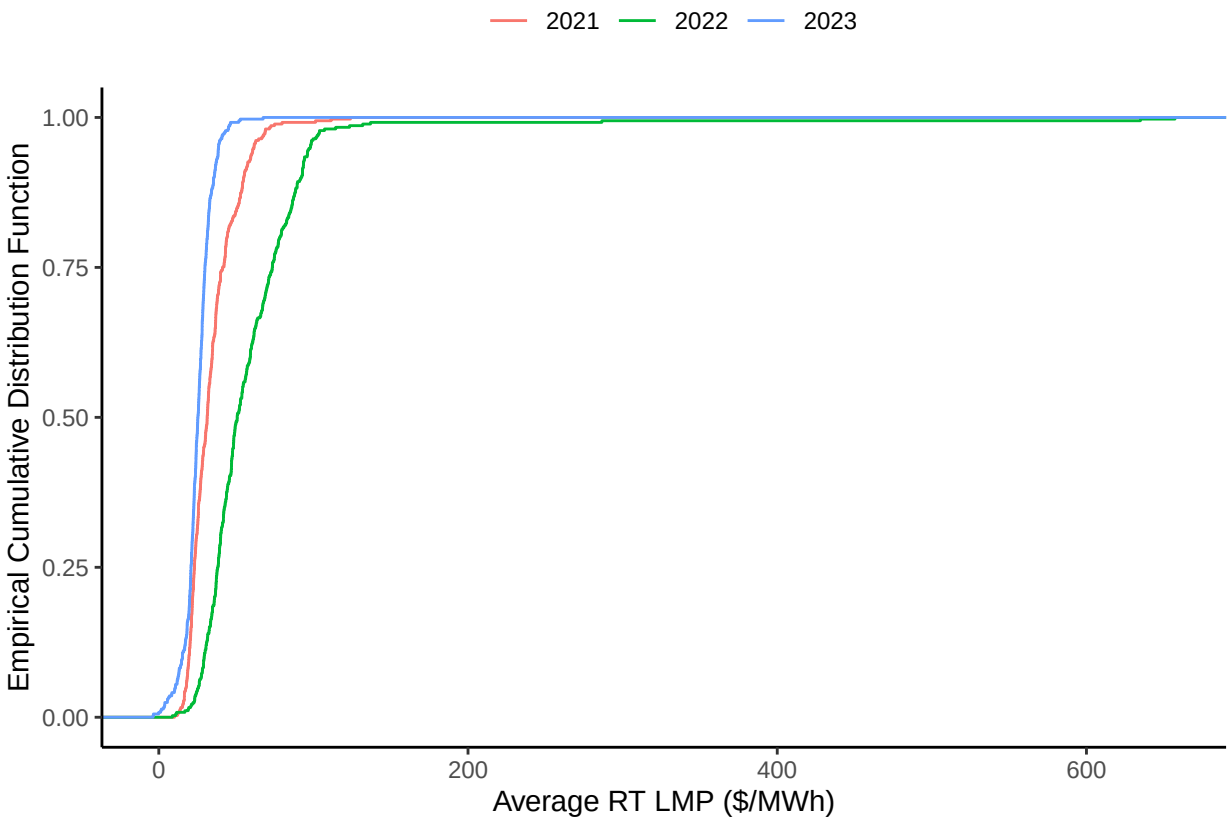



```
data_CGH_long$month_name <- factor(data_CGH_long$month, levels =
  c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"))
ggplot(data_CGH_long, aes(x=month_name,y=values, color = factor(lmp_type))) +
        stat_summary() +
        labs(color = '', x = '', y = 'LMP ($/MWh)') +
        coord_cartesian(ylim = c(0, 100)) +
        scale_y_continuous(expand = c(0,0)) +
        theme_classic() +
        theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
        facet_wrap(~year)
```

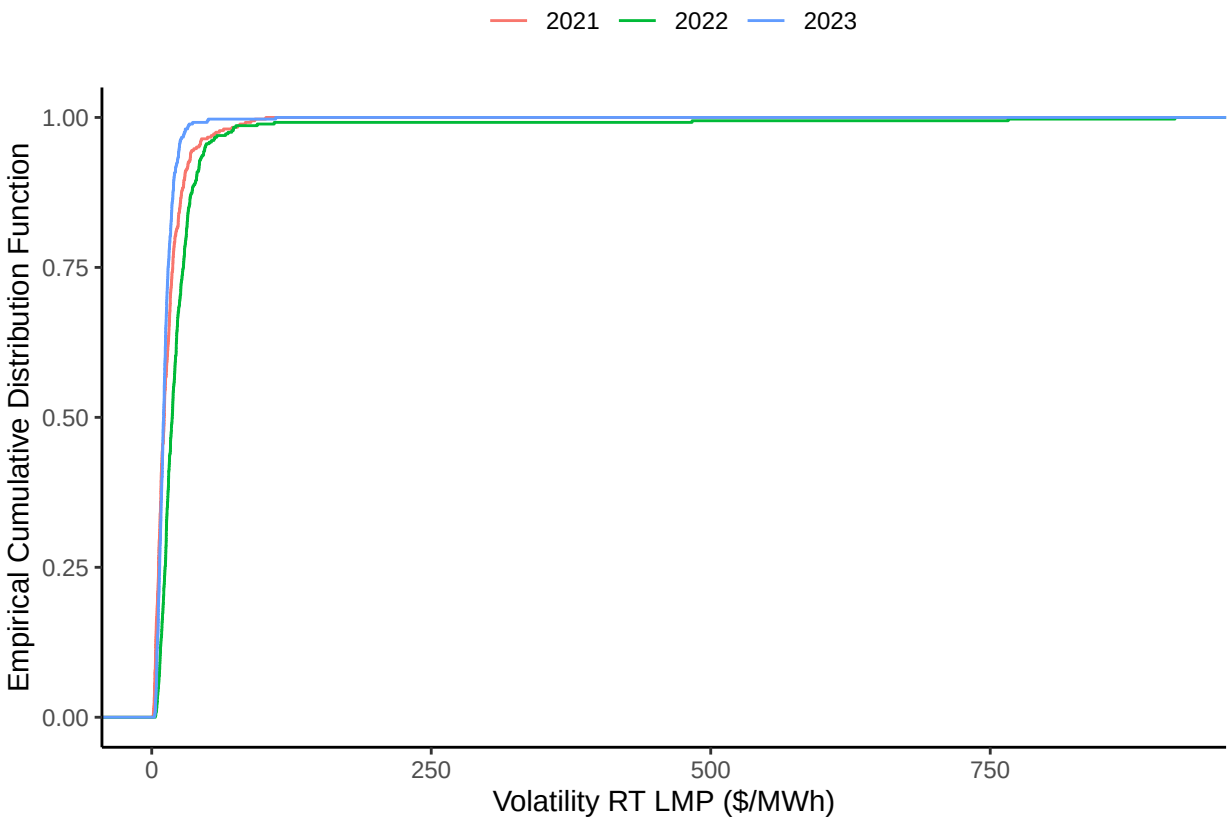
```
## No summary function supplied, defaulting to `mean_se()`
## No summary function supplied, defaulting to `mean_se()`
## No summary function supplied, defaulting to `mean_se()`
```



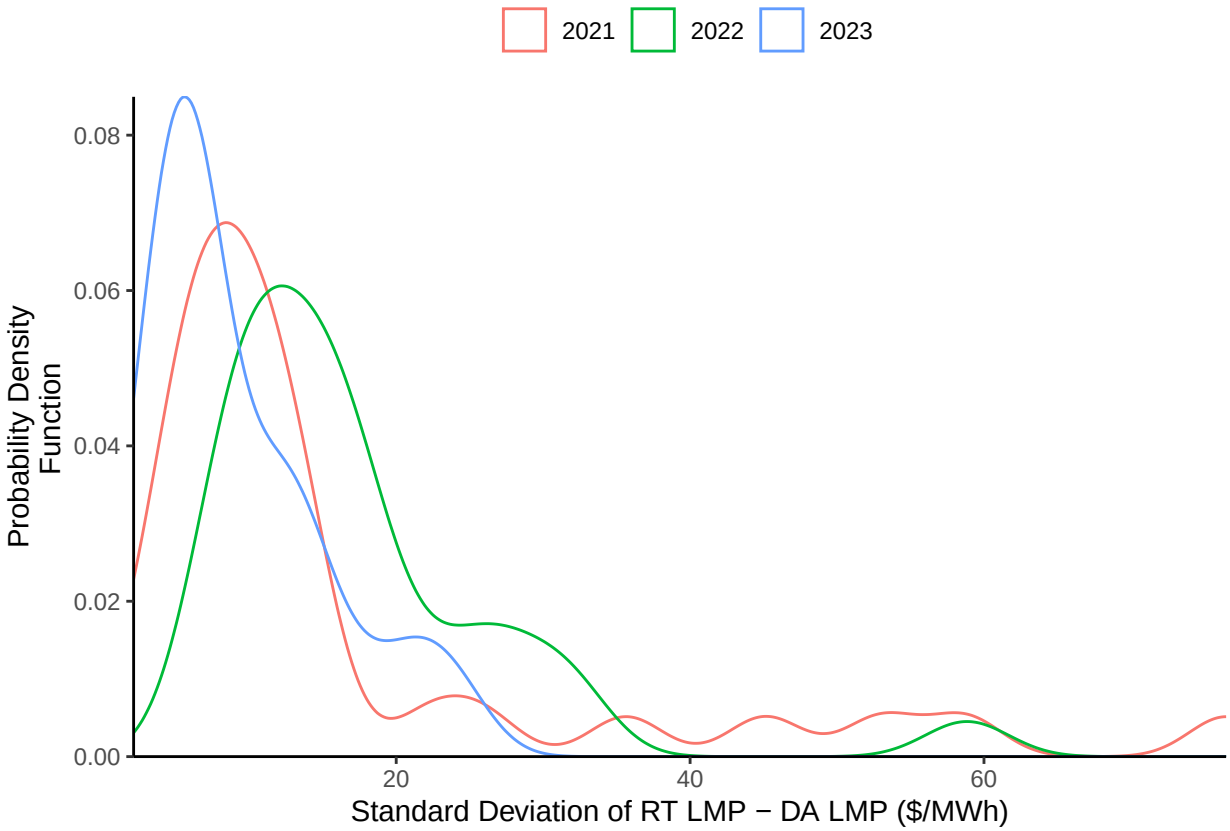

```
CGH_summary_daily <- data_CGH[, .(mean_rt_lmp = mean(total_lmp_rt),
  mean_da_lmp = mean(total_lmp_da)),
  by = .(day, month,year)]
ggplot(CGH_summary_daily, aes(mean_rt_lmp, color = factor(year))) +
    stat_ecdf() +
    labs(color = '', x = 'Average RT LMP ($/MWh)',
    y = 'Empirical Cumulative Distribution Function') +
    theme_classic() +
    theme(legend.position = 'top')
```




```
CGH_summary_daily <- data_CGH[, .(vol_lmp_rt = sd(total_lmp_rt),
  vol_lmp_da = sd(total_lmp_da)),
  by = .(day, month,year)]
ggplot(CGH_summary_daily, aes(vol_lmp_rt, color = factor(year))) +
    stat_ecdf() +
    labs(color = '', x = 'Volatility RT LMP ($/MWh)',
    y = 'Empirical Cumulative Distribution Function') +
    theme_classic() +
    theme(legend.position = 'top')
```

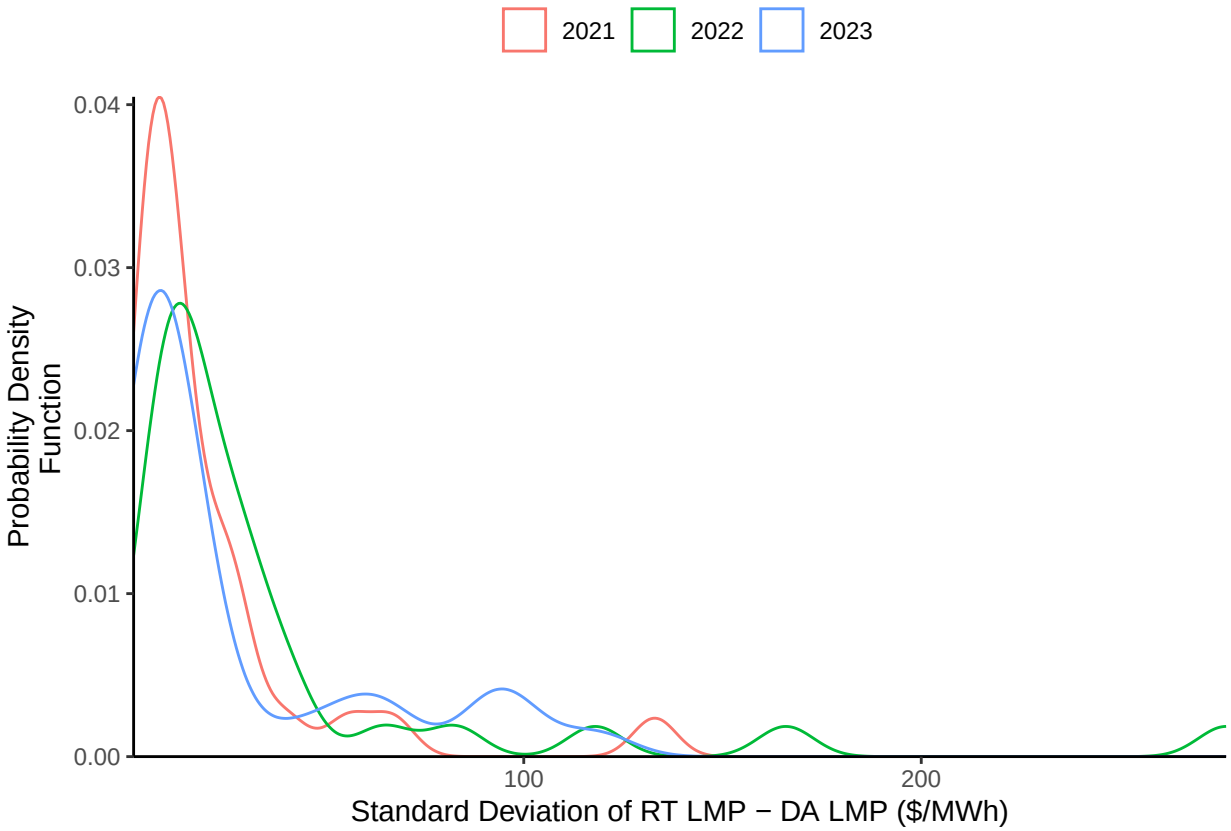



```
vol_daily_CGH <- data_CGH[, .(vol_spread = sd(spread, na.rm = TRUE)),
by = .(day, month, year)]
  ggplot(vol_daily_CGH[month == 'Aug'], aes(x=vol_spread, color = factor(year))) +
  geom_density() +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)', y = 'Probability Density
  Function', color = '') +
  scale_x_continuous(expand = c(0,0)) +
  scale_y_continuous(expand = c(0,0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



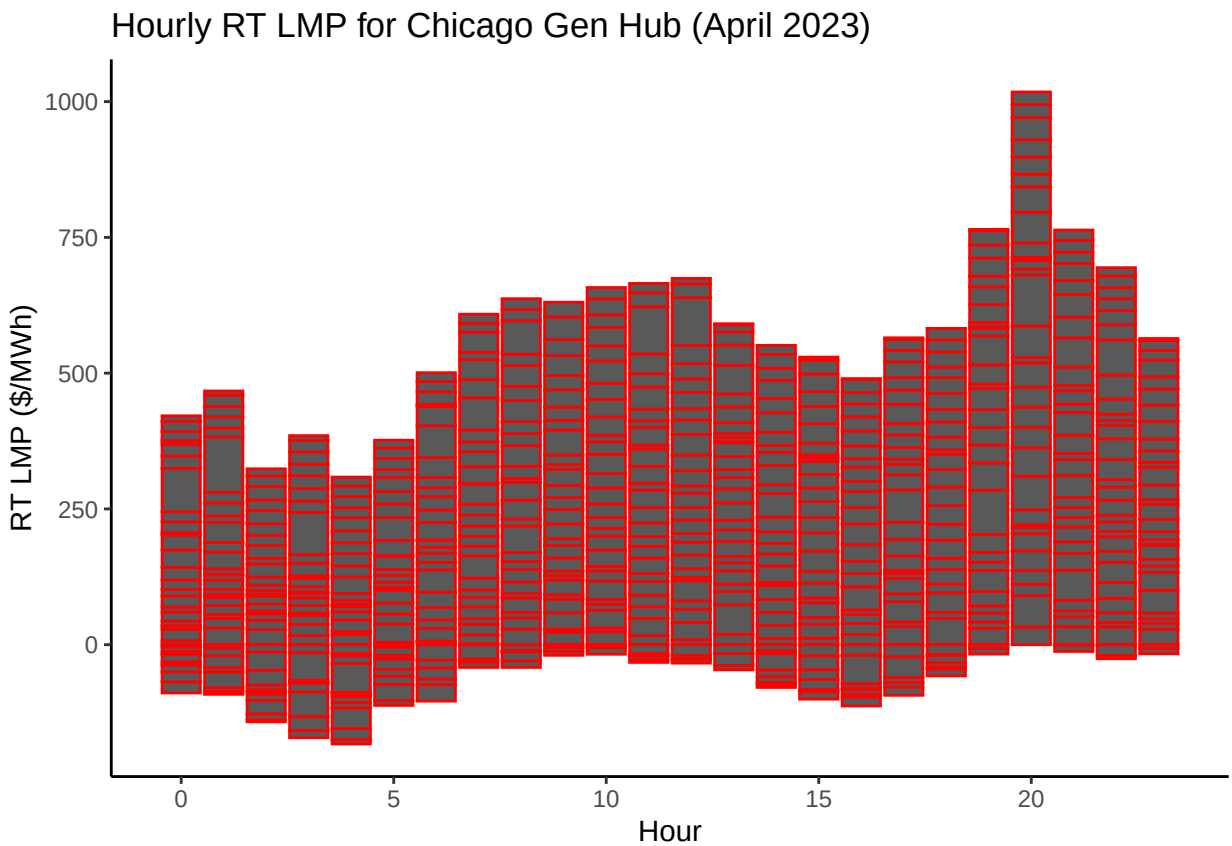

```
#We are only looking at August for each year. The vol spread is calculated as RT- DA. 2022 has the wide
```

```
#Comparing to the eastern Hub

#1 creating datatable
data_easternHub <- data_PJM[pnode_name == 'EASTERN HUB']
#2 calculating the vol spread
vol_daily_EH <- data_easternHub[, .(vol_spread = sd(spread, na.rm = TRUE)),
by = .(day, month, year)]
#3 Plotting
  ggplot(vol_daily_EH[month == 'Aug'], aes(x=vol_spread, color = factor(year))) +
  geom_density() +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)', y = 'Probability Density
  Function', color = '') +
  scale_x_continuous(expand = c(0,0)) +
  scale_y_continuous(expand = c(0,0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



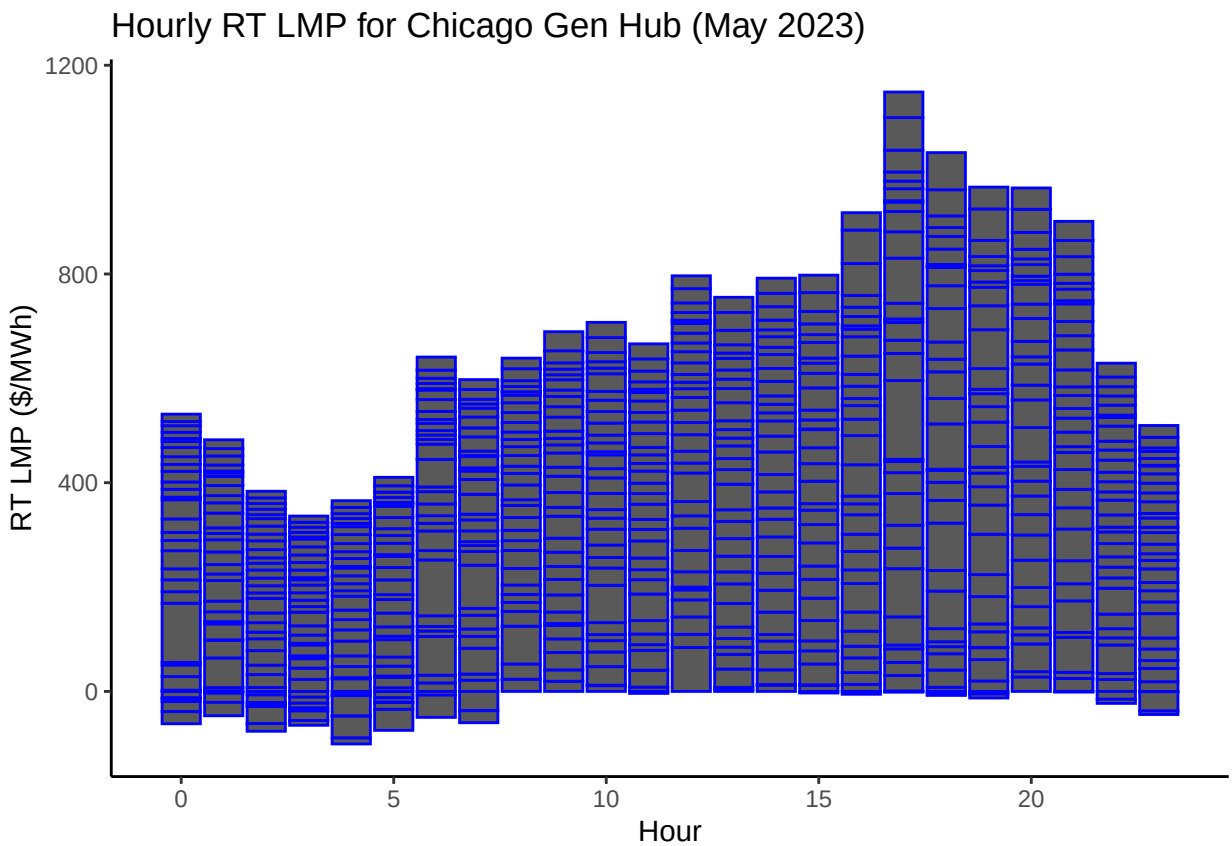

```
#We see tighter spreads, and lower volatility for the Eastern Hub
```

```
#Plot the hourly by day for the month of April 2023 for Chicago GEN hum on RT LMP

# Filter April 2023
april_data <- data_CGH[month == 'Apr' & year == 2023]


ggplot(april_data, aes(x = hour, y = total_lmp_rt)) +
  geom_col(color = "red") +
  labs(title = "Hourly RT LMP for Chicago Gen Hub (April 2023)",
       x = "Hour", y = "RT LMP ($/MWh)") +
  theme_classic()
```

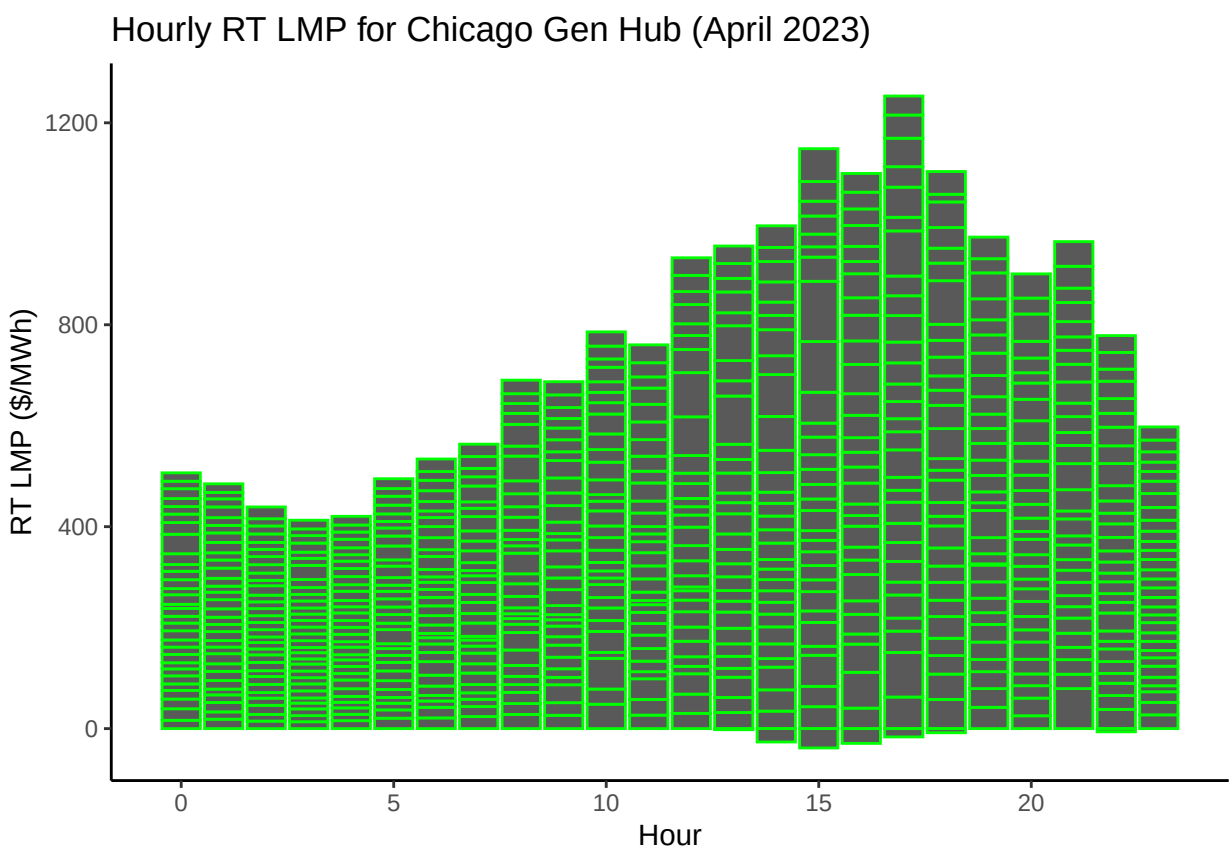



```
#Negative LMP prices indicate that there was a greater supply than demand of energy. Since we had a sur
```

```
# Filter May 2023
library(dplyr)

may_data <- data_CGH[month == 'May' & year == 2023]


ggplot(may_data, aes(x = hour, y = total_lmp_rt)) +
  geom_col(color = "blue") +
  labs(title = "Hourly RT LMP for Chicago Gen Hub (May 2023)",
       x = "Hour", y = "RT LMP ($/MWh)") +
  theme_classic()
```




```r
# Filter June 2023
june_data <- data_CGH[month == 'Jun' & year == 2023]


ggplot(june_data, aes(x = hour, y = total_lmp_rt)) +
  geom_col(color = "green") +
  labs(title = "Hourly RT LMP for Chicago Gen Hub (April 2023)",
       x = "Hour", y = "RT LMP ($/MWh)") +
  theme_classic()
```



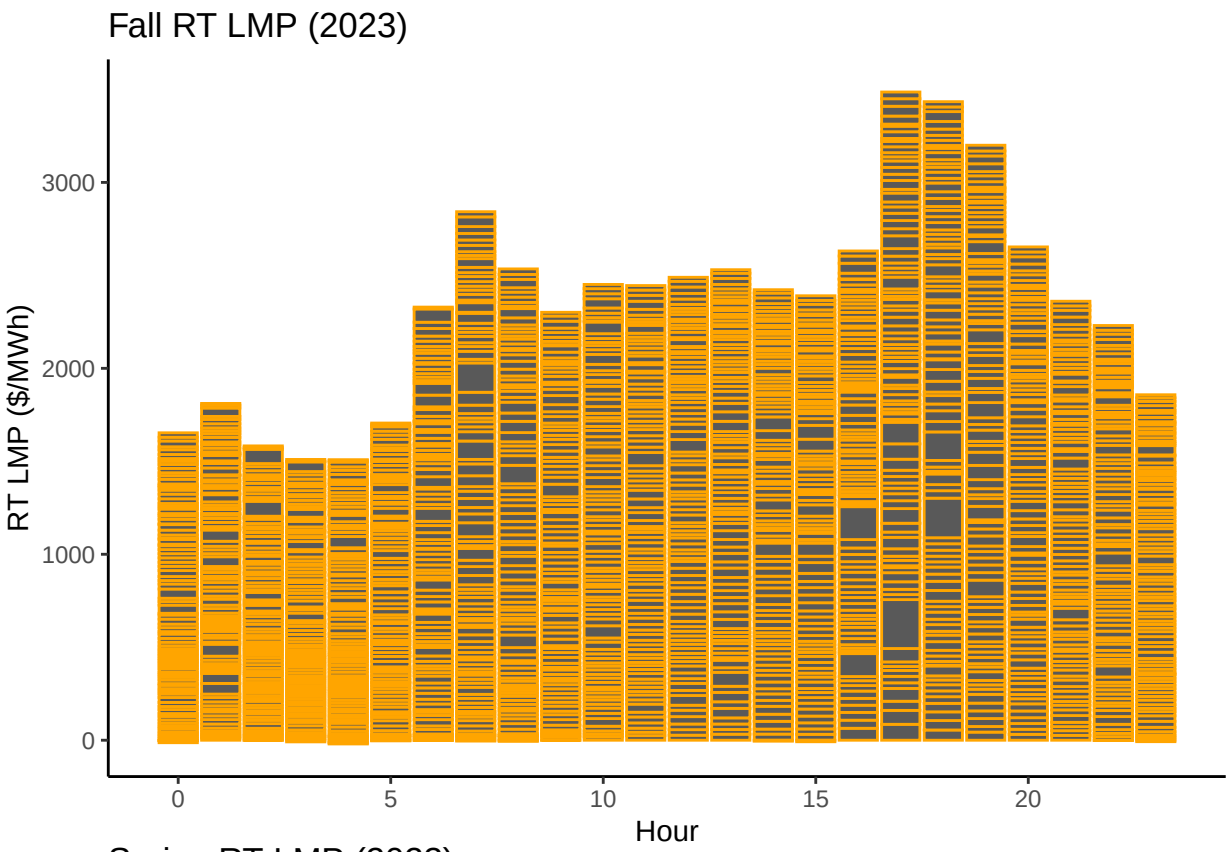

```r
library(dplyr)
library(purrr)
```

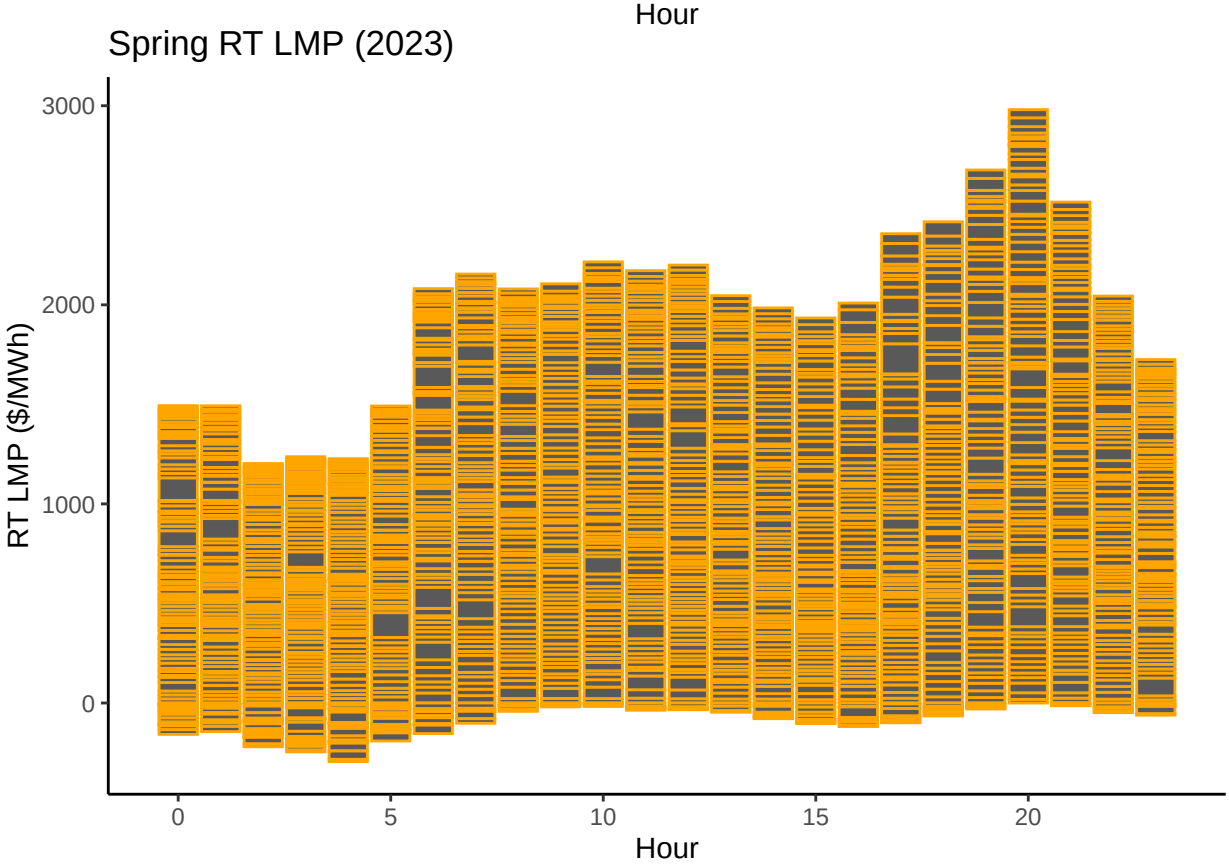
```
##
## Attaching package: 'purrr'

## The following object is masked from 'package:data.table':
##
##     transpose
```

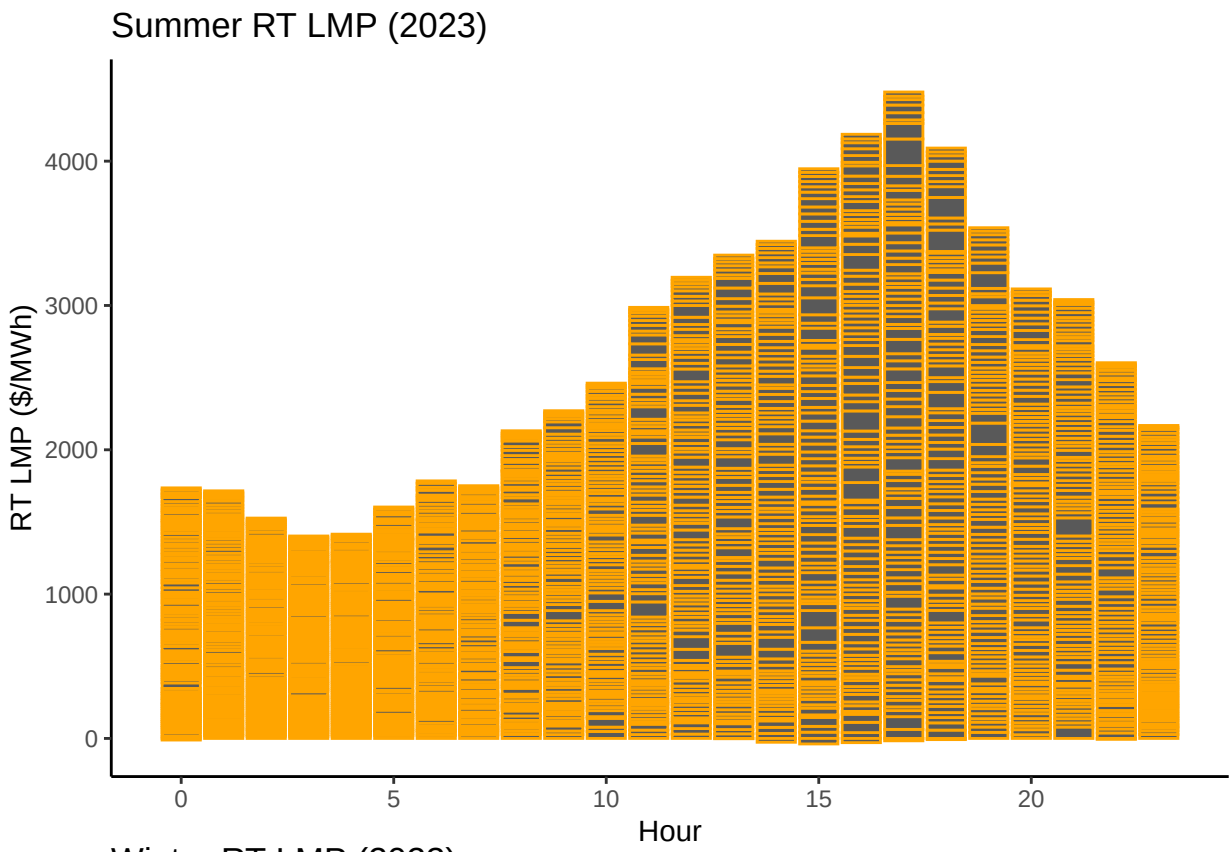
```r
mapping <- list(
  "Jan" = "Winter",
  "Feb" = "Winter",
  "Mar" = "Spring",
  "Apr" = "Spring",
  "May" = "Spring",
  "Jun" = "Summer",
  "Jul" = "Summer",
  "Aug" = "Summer",
  "Sep" = "Fall",
  "Oct" = "Fall",
  "Nov" = "Fall",
  "Dec" = "Winter"
)

# Apply the mapping
data_CGH <- data_CGH %>%
  mutate(
```

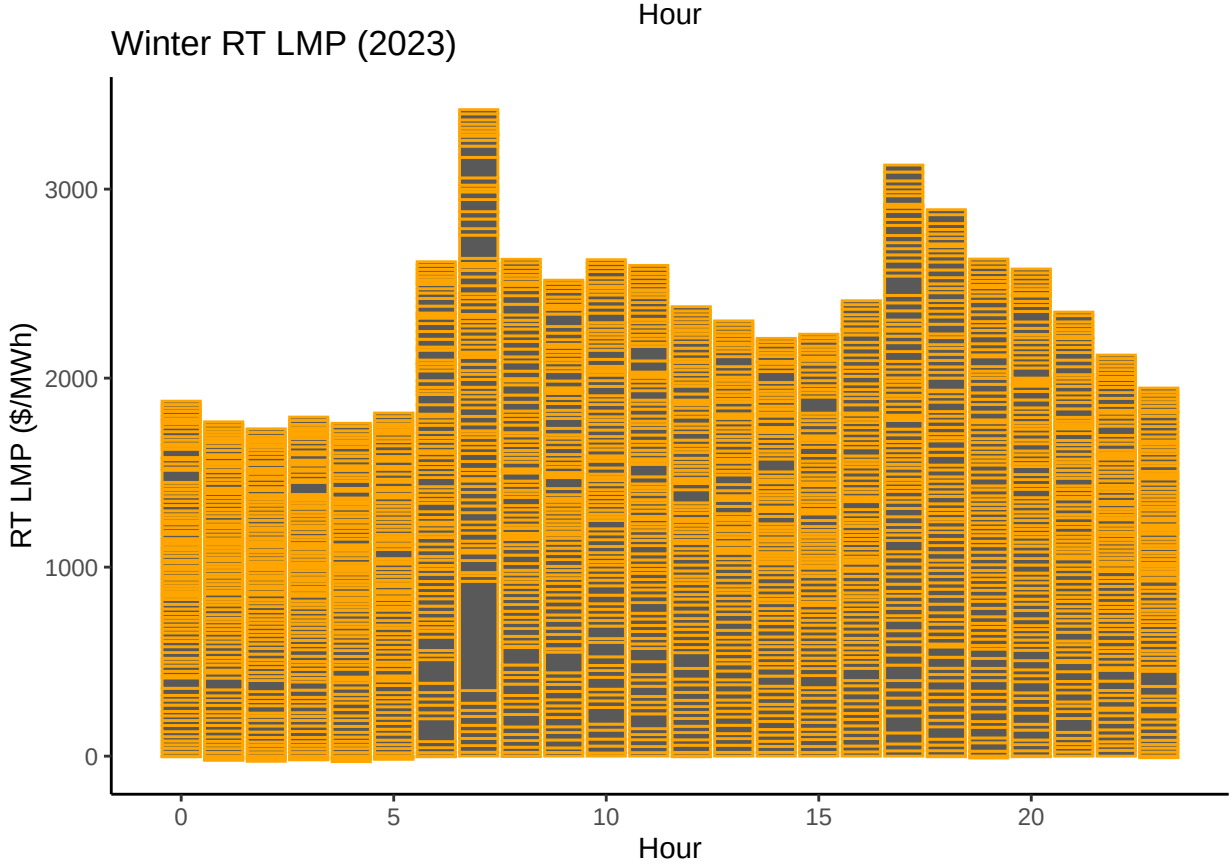
```r
    season = map_chr(month, ~ mapping[[.x]])
  )
```

```r
# Filter April 2023

library(dplyr)
library(ggplot2)

library(dplyr)
library(ggplot2)

# Filter data for 2023
data_2023_cgh <- data_CGH %>% filter(year == 2023)

# Split data by season
season_list <- split(data_2023_cgh, data_2023_cgh$season)

plot_season_lmp <- function(df, season_name) {
  ggplot(df, aes(x = hour, y = total_lmp_rt)) +
    geom_col(color = "orange") +
    labs(title = paste(season_name, "RT LMP (2023)"),
         x = "Hour", y = "RT LMP ($/MWh)") +
    theme_classic()
}

season_plots <- lapply(names(season_list), function(season_name) {
  plot_season_lmp(season_list[[season_name]], season_name)
})

for (p in season_plots) print(p)
```


Fall RT LMP (2023)
RT LMP ($/MWh)
3000
2000
1000
0
0
5
10
15
20
Hour

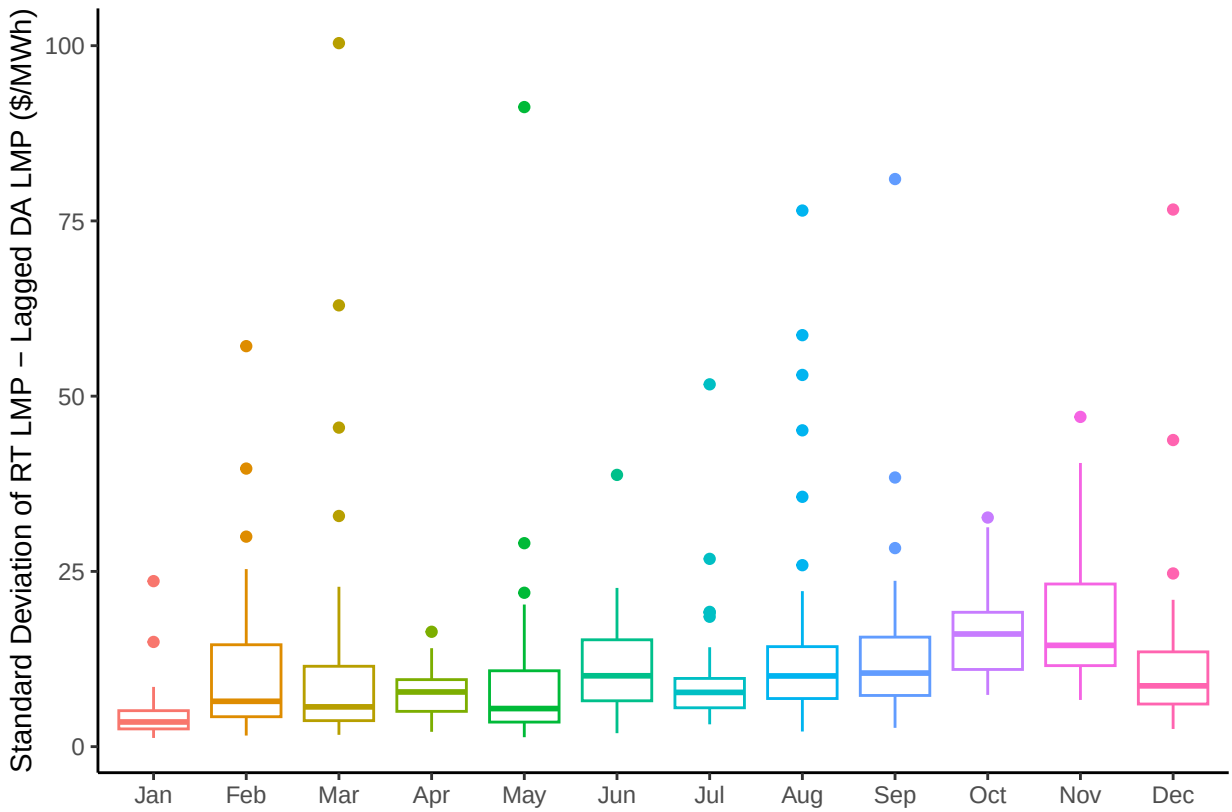

Spring RT LMP (2023)
RT LMP ($/MWh)
3000
2000
1000
0
0
5
10
15
20
Hour


Summer RT LMP (2023)
RT LMP ($/MWh)
4000
3000
2000
1000
0
0
5
10
15
20
Hour

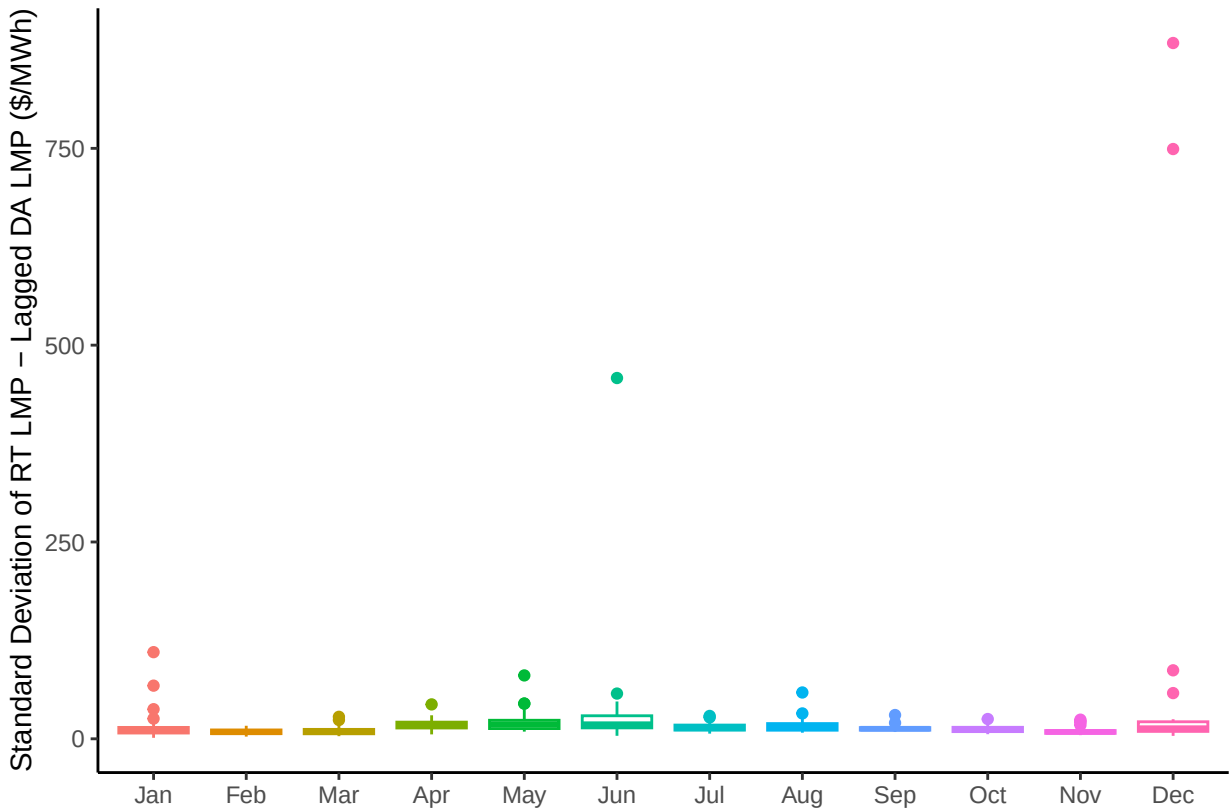

Winter RT LMP (2023)
RT LMP ($/MWh)
3000
2000
1000
0
0
5
10
15
20
Hour

```r
#based on the above graph it looks like the highest is around 8:00 PM and the lowest is around 4:00 AM
```

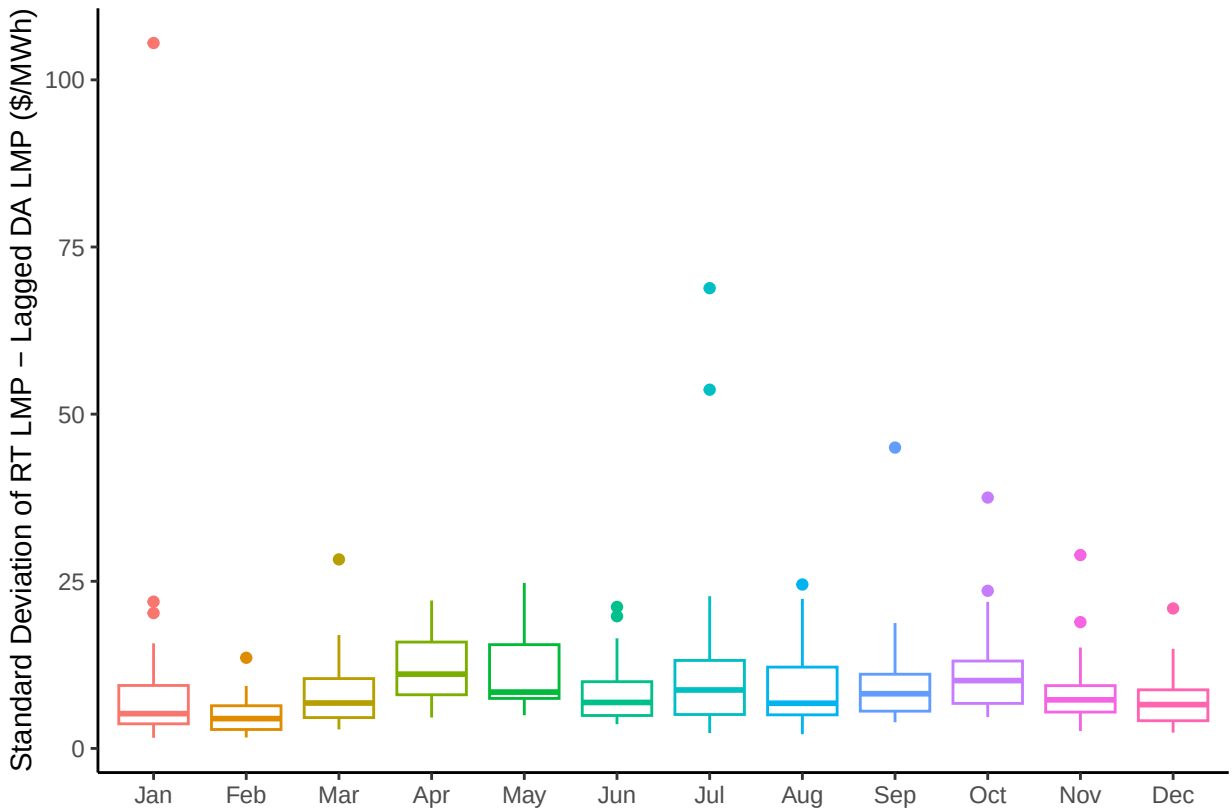
```r
#daily bocplots of RT-DA LMP Spread for 2021
ggplot(vol_daily_CGH[year == 2021], aes(factor(month, levels = month.abb),
vol_spread,
color = factor(month, levels = month.abb))) +
geom_boxplot() +
labs(x = '', y = 'Standard Deviation of RT LMP - Lagged DA LMP ($/MWh)') +
theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust= 1)) +
guides(color ='none') +
theme_classic() +
theme(legend.position = 'top')
```




```r
#2022
ggplot(vol_daily_CGH[year == 2022], aes(factor(month, levels = month.abb),
vol_spread,
color = factor(month, levels = month.abb))) +
geom_boxplot() +
labs(x = '', y = 'Standard Deviation of RT LMP - Lagged DA LMP ($/MWh)') +
theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust= 1)) +
guides(color ='none') +
theme_classic() +
theme(legend.position = 'top')
```




```
#2023
ggplot(vol_daily_CGH[year == 2023], aes(factor(month, levels = month.abb),
vol_spread,
color = factor(month, levels = month.abb))) +
geom_boxplot() +
labs(x = '', y = 'Standard Deviation of RT LMP - Lagged DA LMP ($/MWh)') +
theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust= 1)) +
guides(color ='none') +
theme_classic() +
theme(legend.position = 'top')
```




```r
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:data.table':
##
##     hour, isoweek, mday, minute, month, quarter, second, wday, week,
##     yday, year

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

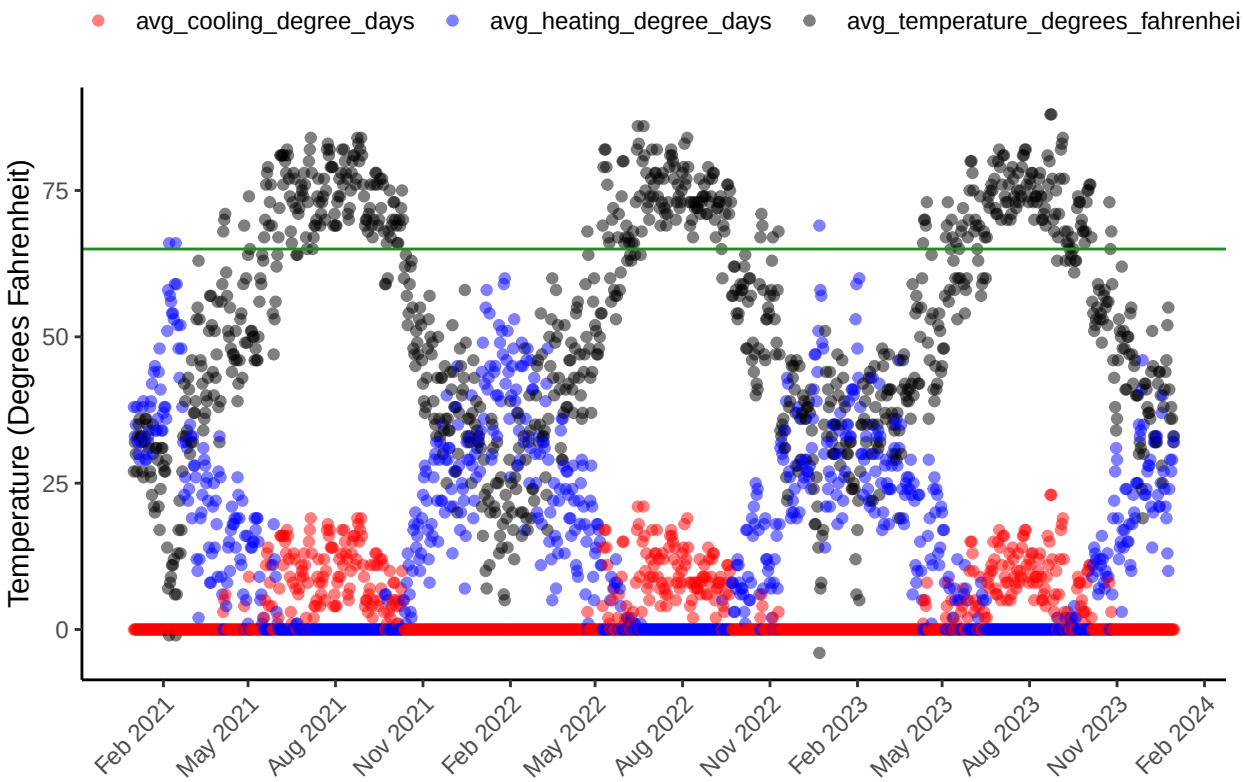
```r
# Assuming 'vol_daily_CGH' has columns: year, month, day, vol_spread
# If 'month' is a character like "Jan", convert it to numeric
vol_daily_CGH_wide <- vol_daily_CGH %>%
  mutate(
    month_num = match(month, month.abb),  # Converts "Jan" to 1, "Feb" to 2, etc.
    date = make_date(year, month_num, day)
  ) %>%
  select(date, vol_spread)  # Keep only the columns you want
```

```r
vol_daily_CGH_wide %>%
  filter(vol_spread == max(vol_spread, na.rm = TRUE))
```

```
## Indices: <month>, <year>
##          date vol_spread
##        <Date>      <num>
## 1: 2022-12-23   883.7946
```

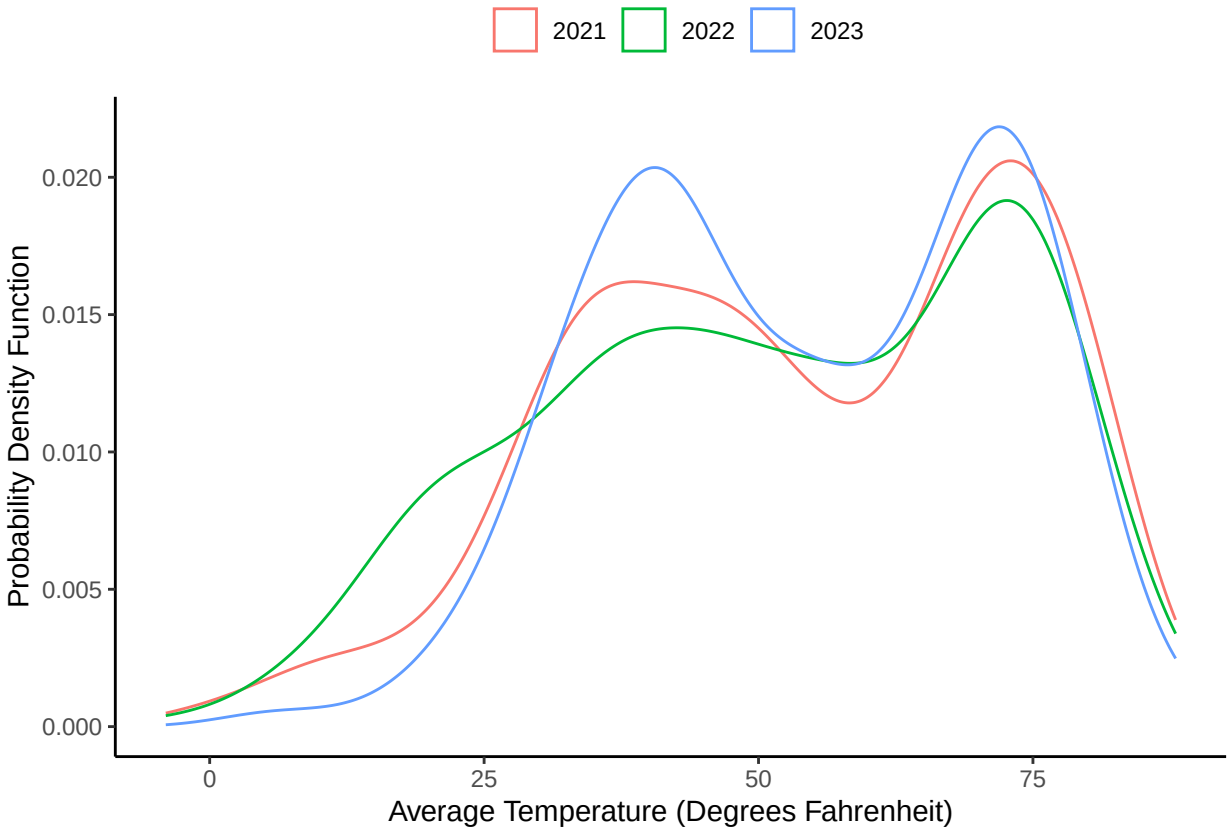
```r
# 2022-12-2023 is the date with the most volatility
```

```r
library(dplyr)
library(tidyr)
library(data.table)
library(ggplot2)

daily_temp_Chicago <- data_Chicago[, .(date, month, year,
                                       avg_temperature_degrees_fahrenheit,
                                       avg_cooling_degree_days,
                                       avg_heating_degree_days)] %>%
  unique() %>%
  pivot_longer(
    cols = starts_with('avg'),
    names_to = 'temp_series',
    values_to = 'degrees'
  )

setDT(daily_temp_Chicago)

ggplot(daily_temp_Chicago, aes(date, degrees, color = factor(temp_series))) +
  geom_point(alpha = 0.5) +
  scale_color_manual(values = c('red','blue','black')) +
  geom_hline(yintercept = 65, color = 'forestgreen') +
  labs(
    x = '',
    color = '',
    y = 'Temperature (Degrees Fahrenheit)'
  ) +
  scale_x_date(date_breaks = '3 months', date_labels = "%b %Y") +
  theme_classic() +
  theme(
    legend.position = 'top',
    axis.text.x = element_text(angle = 45, vjust = 1, hjust= 1)
  )
```



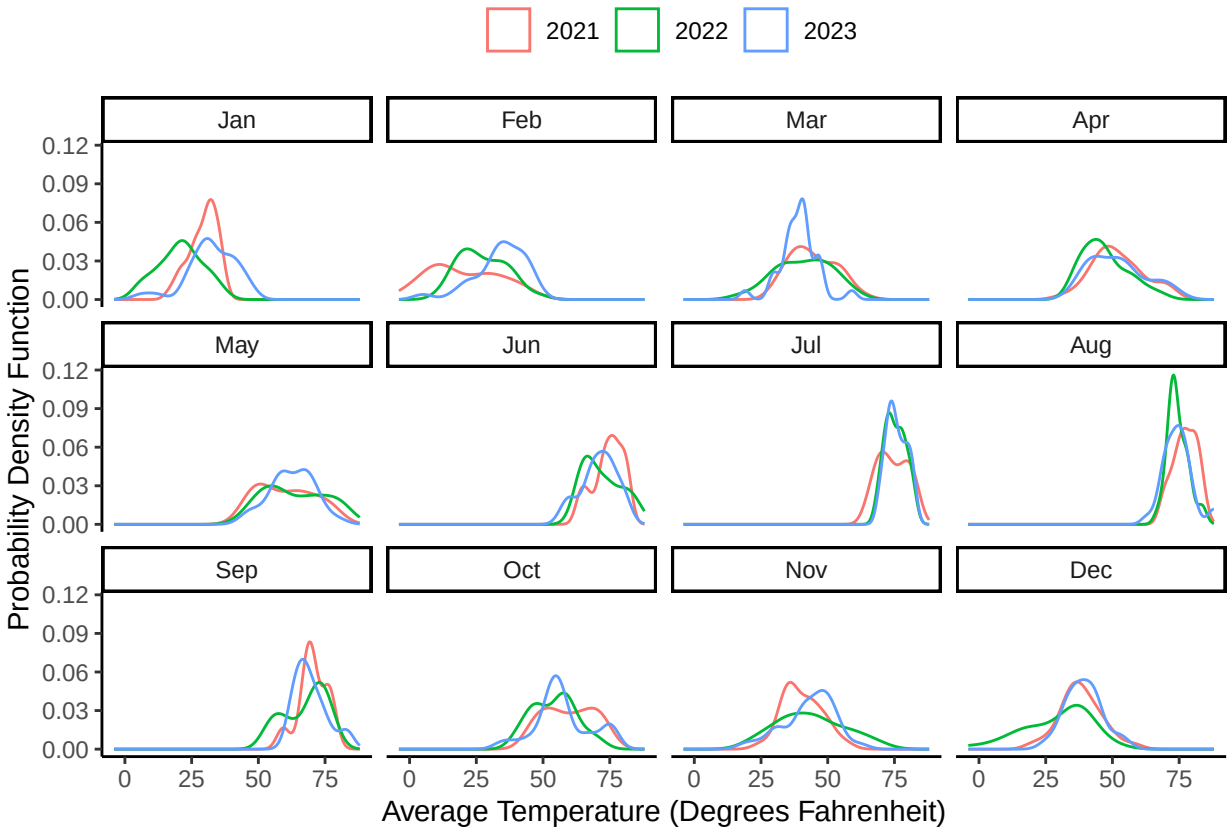

```
#There is a cyclical weather pattern. From a glance it looks like there are more heating days then cool
```

```
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
aes(degrees, color = factor(year))) +
geom_density() +
labs(color ='', x = 'Average Temperature (Degrees Fahrenheit)',
y = 'Probability Density Function') +
theme_classic() +
theme(legend.position = 'top')
```



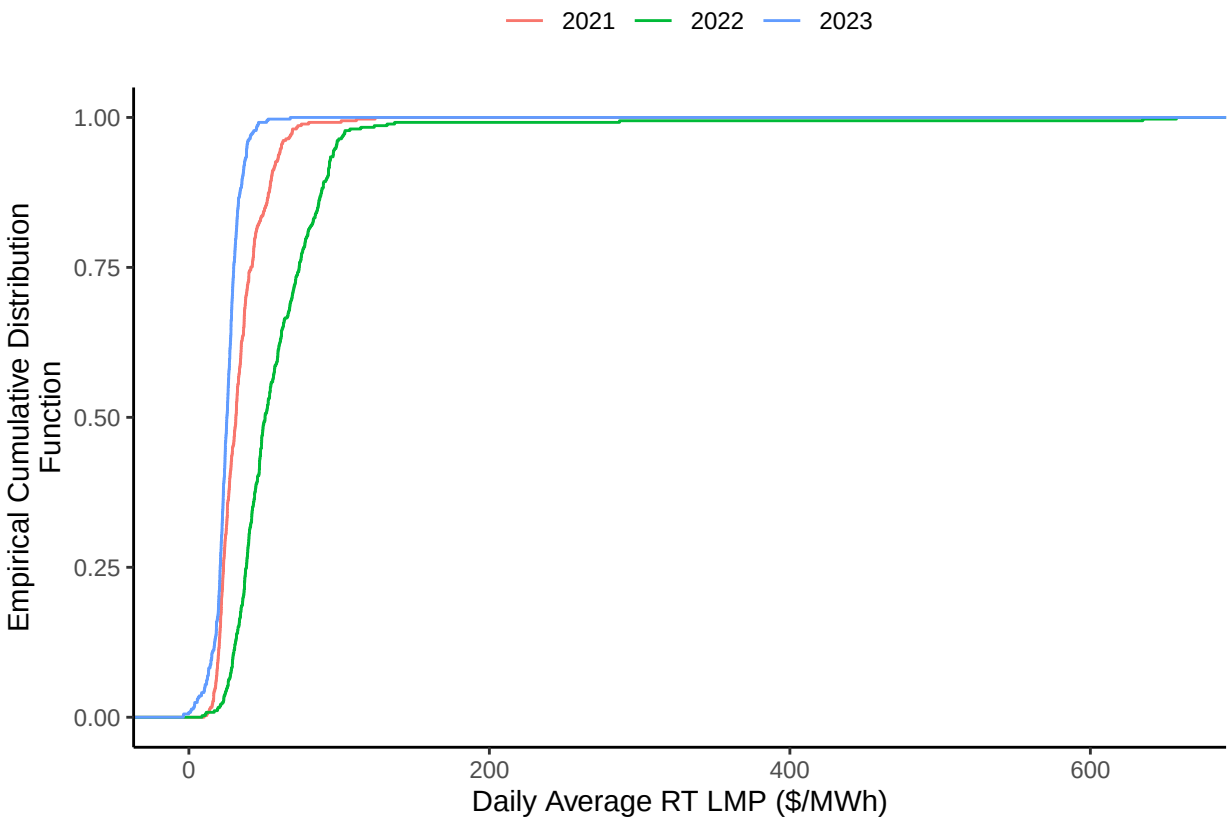

```
#We notice the shape of the distribution to be skewed to the left. Out of the three years, 2022 looks l
```

```
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
       aes(degrees, color = factor(year))) +
  geom_density() +
  labs(color = '',
       x = 'Average Temperature (Degrees Fahrenheit)',
       y = 'Probability Density Function') +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~factor(month, levels = month.abb))
```

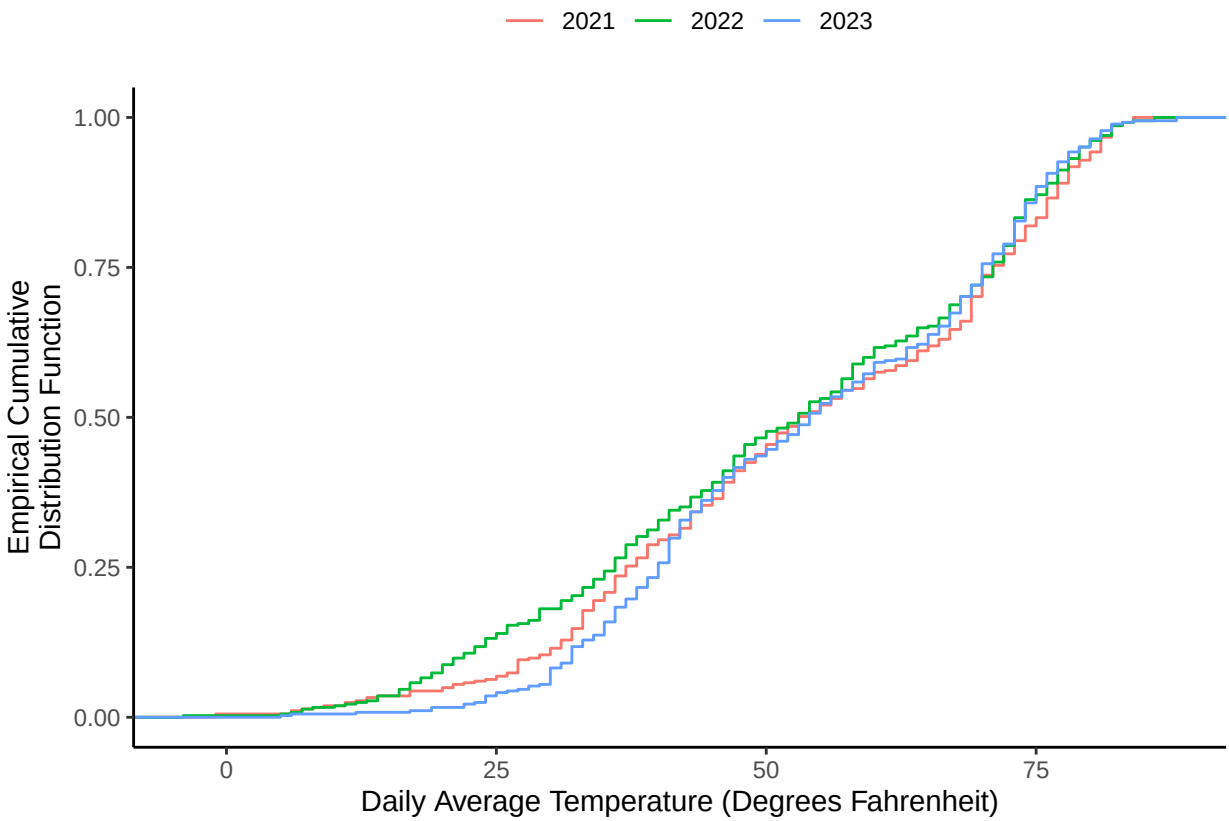



```
#this lines up with our hypothesis of the bimodal model, since we see peaks in the winter seasons and s
```

```
lmp_temp_Chicago <- data_Chicago[,.(mean_lmp_rt = mean(total_lmp_rt),
    mean_lmp_da = mean(total_lmp_da),
    vol_spread = sd(spread),
    temp = mean(avg_temperature_degrees_fahrenheit),
    humidity = mean(relative_humidity_percent)),
  by = .(date, month, year)]

ggplot(lmp_temp_Chicago, aes(mean_lmp_rt, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average RT LMP ($/MWh)', y = 'Empirical Cumulative Distribution
Function',
  color = '') +
theme_classic() +
theme(legend.position = 'top')
```




```
ggplot(lmp_temp_Chicago, aes(temp, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average Temperature (Degrees Fahrenheit)', y = 'Empirical Cumulative
Distribution Function',
  color = '') +
  theme_classic() +
  theme(legend.position = 'top')
```

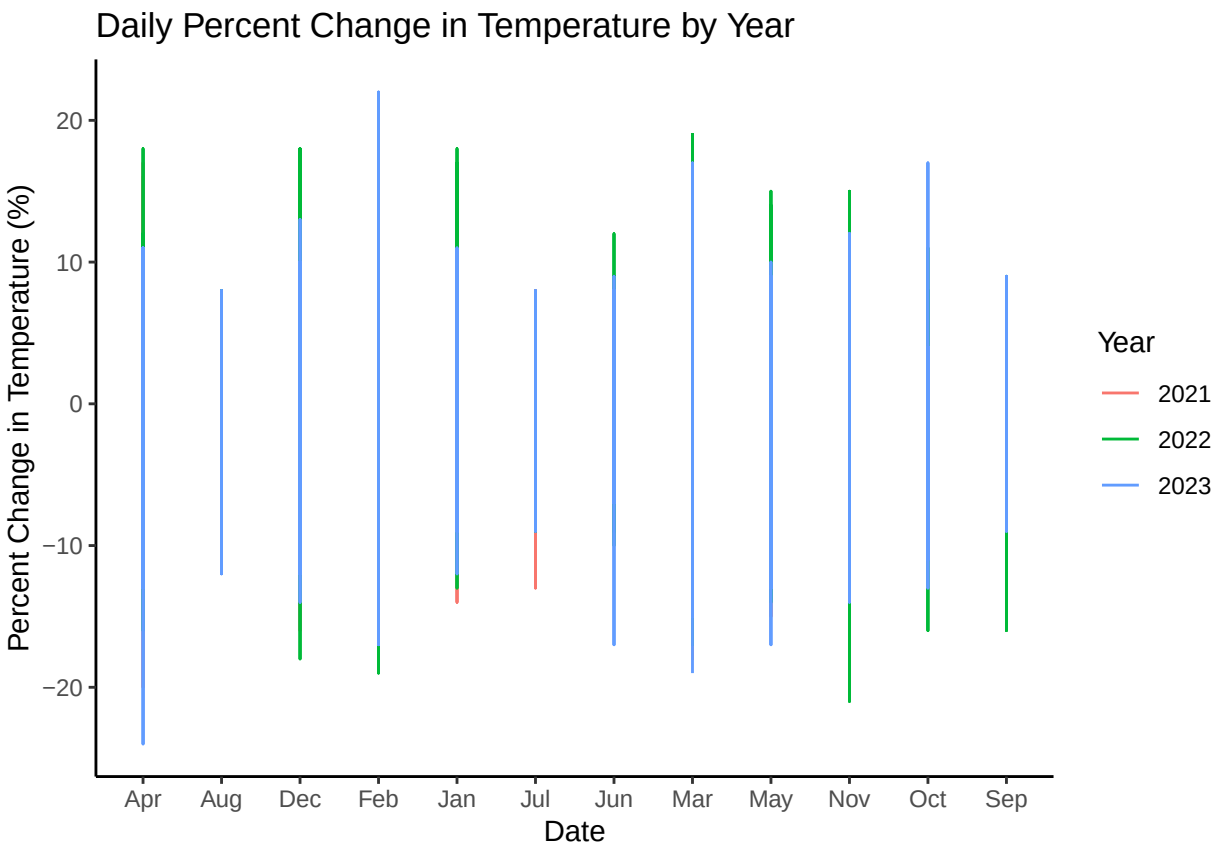



```
#As stated above there is a clear FOSD with 2022 utilizing the highest daily average RT LMP. This is no
```

```
data_Chicago[, change_temp := shift(avg_temperature_degrees_fahrenheit, type = "lead") - avg_temperatur
```

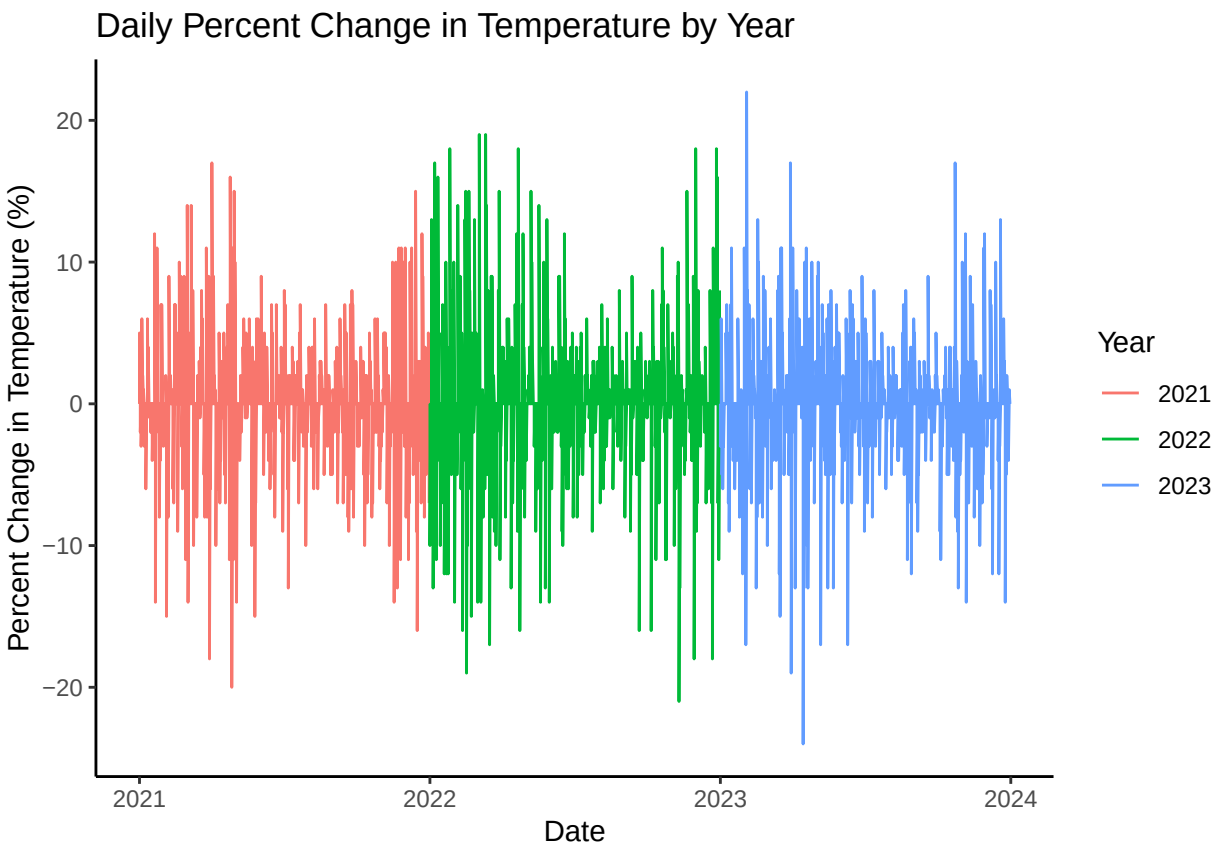
```
ggplot(data_Chicago, aes(x = month, y = change_temp, color = factor(year))) +
  geom_line() +
  labs(title = "Daily Percent Change in Temperature by Year",
       x = "Date",
       y = "Percent Change in Temperature (%)",
       color = "Year") +
  theme_classic()
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```




```r
ggplot(data_Chicago, aes(x = date, y = change_temp, color = factor(year))) +
  geom_line() +
  labs(title = "Daily Percent Change in Temperature by Year",
       x = "Date",
       y = "Percent Change in Temperature (%)",
       color = "Year") +
  theme_classic()
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```




```
years <- c(2021, 2022, 2023)

for (y in years) {
  sd_val <- data_Chicago[year == y, sd(avg_temperature_degrees_fahrenheit)]
  print(paste("Year:", y, "SD:", sd_val))
}
```

```
## [1] "Year: 2021 SD: 19.7821463164315"
## [1] "Year: 2022 SD: 20.651650568102"
## [1] "Year: 2023 SD: 17.5274362608774"
```

```
#We say the highest volatility in the average daily change in temp in 2022, which helps explain the inc
```

When you save the notebook, an HTML file containing the code and output will be saved alongside it (click the *Preview* button or press *Cmd+Shift+K* to preview the HTML file).

The preview shows you a rendered HTML copy of the contents of the editor. Consequently, unlike *Knit*, *Preview* does not run any R code chunks. Instead, the output of the chunk when it was last run in the editor is displayed.